

Temporal Planning Semantics

Mohammad Abdulaziz Lukas Koller

This is a formalisation of the semantics of temporal planning as used in the Planning Domain Definition Language. Based on that, we also develop a plan validator which is verified w.r.t. those semantics. It is based on an earlier formalisation of the semantics of classical planning [2]. This work was described in an ealier publication [1].

Contents

1	Temporal PDDL Semantics	3
1.1	Utility Functions	3
1.2	Abstract Syntax	3
1.2.1	Generic Entities	3
1.2.2	Domains	4
1.2.3	Problems	5
1.2.4	Plans	5
1.2.5	Ground Actions	6
1.3	Closed-World Assumption, Equality, and Negation	6
1.3.1	Proper Generalization	8
1.4	Happening Execution Semantics	8
1.5	Well-Formedness of PDDL	9
1.6	PDDL Semantics	14
1.7	Preservation of Well-Formedness	20
1.7.1	Well-Formed Action Instances	20
1.7.2	Preservation	23
1.7.3	Completeness of Semantics	26
2	Executable Temporal PDDL Checker	32
2.1	Generic DFS Reachability Checker	32
2.2	Implementation Refinements	34
2.2.1	Of-Type	34
2.2.2	Ground Action Interference & Application of Effects	37
2.2.3	Well-Formedness	38
2.2.4	Happening Execution	40
2.2.5	Construction of Induced Happening Sequence	41
2.3	Executable Plan Checker	58

2.4	Code Setup	60
2.4.1	Code Equations	60
2.4.2	Setup for Containers Framework	62
2.4.3	More Efficient Distinctness Check for Linorders	62
2.4.4	Parsing Rational Numbers	63
2.4.5	Code Generation	64
3	Connecting the Semantics to State-Transition Based Semantics	65
3.1	State-Transition Based Semantics	66
3.2	Connecting the State-Transition Based Semantics to Happening Sequence Based Semantics	68

1 Temporal PDDL Semantics

```
theory TEMPORAL-PDDL-Semantics
imports
  Propositional-Proof-Systems.Formulas
  Propositional-Proof-Systems.Sema
  Propositional-Proof-Systems.Consistency
  Automatic-Refinement.Misc
  Automatic-Refinement.Refine-Util
  HOL.Rat
```

```
begin
no-notation insert (- ▷ - [56,55] 55)
```

1.1 Utility Functions

```
abbreviation strict-sorted::'a::{linorder} list  $\Rightarrow$  bool where strict-sorted  $\equiv$  sorted-wrt (<)
```

```
definition index-by f l  $\equiv$  map-of (map ( $\lambda x.$  (f x,x)) l)
```

```
lemma index-by-eq-Some-eq[simp]:
  assumes distinct (map f l)
  shows index-by f l n = Some x  $\longleftrightarrow$  ( $x \in \text{set } l \wedge f x = n$ )
   $\langle \text{proof} \rangle$ 
```

```
lemma index-by-eq-SomeD:
  shows index-by f l n = Some x  $\implies$  ( $x \in \text{set } l \wedge f x = n$ )
   $\langle \text{proof} \rangle$ 
```

```
lemma lookup-zip-idx-eq:
  assumes length params = length args
  assumes i < length args
  assumes distinct params
  assumes k = params ! i
  shows map-of (zip params args) k = Some (args ! i)
   $\langle \text{proof} \rangle$ 
```

```
lemma rtrancl-image-idem[simp]:  $R^* \circ R^* \circ s = R^* \circ s$ 
   $\langle \text{proof} \rangle$ 
```

1.2 Abstract Syntax

1.2.1 Generic Entities

Defines the time number type

```
type-synonym time = rat
```

type-synonym *name* = *string*

datatype *predicate* = *Pred* (*name*: *name*)

datatype *func* = *Func* (*name*: *name*)

Some of the AST entities are defined over a polymorphic *'val* type, which gets either instantiated by variables (for domains) or objects (for problems).

An atom is either a predicate with arguments, or an equality statement.

datatype *'ent atom* = *predAtm* (*predicate*: *predicate*) (*arguments*: *'ent list*)
| *eqAtm* (*lhs*: *'ent*) (*rhs*: *'ent*)

A type is a list of primitive type names. To model a primitive type, we use a singleton list.

datatype *type* = *Either* (*primitives*: *name list*)

An effect contains a list of values to be added, and a list of values to be removed.

datatype *'ent ast-effect* = *Effect* (*adds*: (*'ent atom formula*) *list*)
(*dels*: (*'ent atom formula*) *list*)

Variables are identified by their names.

datatype *variable* = *Var* (*varname*: *name*)

Objects and constants are identified by their names

datatype *object* = *Obj* (*name*: *name*) | *FuncEnt* (*func*: *func*) (*arguments*: *object list*) | *TimeEnt* *time*

datatype *term* = *VAR* *variable* | *CONST* *object*

hide-const (**open**) *VAR CONST* — Refer to constructors by qualified names only

1.2.2 Domains

A time specifier specifies the time point of conditions and effects in a durative action

datatype *temporal-annotation* = *At-Start* | *Over-All* | *At-End*

datatype *duration-op* = *EQ* | *LEQ* | *GEQ*

datatype *'ent duration-constraint* =
 No-Const
| *Time-Const* (*d-op*: *duration-op*) *time*
| *Func-Const* (*d-op*: *duration-op*) (*func*: *func*) (*arguments*: *'ent list*)

An action schema has a name, a typed parameter list, a precondition, and an effect. In case of a durative action schema there is additionally a duration constraint, and the conditions and effects are timed with a time specifier

```
datatype ast-action-schema =
  Simple-Action-Schema
    (name: name)
    (parameters: (variable  $\times$  type) list)
    (precondition: term atom formula)
    (effect: term ast-effect)
  | Durative-Action-Schema
    (name: name)
    (parameters: (variable  $\times$  type) list)
    (duration-constraint: term duration-constraint)
    (condition: (temporal-annotation  $\times$  (term atom formula)) list)
    (durative-effect: (temporal-annotation  $\times$  (term ast-effect)) list)
```

A predicate declaration contains the predicate's name and its argument types.

```
datatype predicate-decl = PredDecl (pred: predicate) (argTs: type list)
```

```
datatype function-decl = FuncDecl (func: func) (argTs: type list)
```

A domain contains the declarations of primitive types, predicates, and action schemas.

```
datatype ast-domain = Domain
  (types: (name  $\times$  name) list) — (type, supertype) declarations.
  (predicates: predicate-decl list)
  (functions: function-decl list)
  (consts: (object  $\times$  type) list)
  (actions: ast-action-schema list)
```

1.2.3 Problems

A fact is a predicate applied to objects.

```
type-synonym fact = predicate  $\times$  object list
```

A problem consists of a domain, a list of objects, a description of the initial state, and a description of the goal state.

```
datatype ast-problem = Problem
  (domain: ast-domain)
  (objects: (object  $\times$  type) list)
  (init: object atom formula list)
  (goal: object atom formula)
```

1.2.4 Plans

```
datatype plan-action =
```

Simple-Plan-Action (*name: name*) (*arguments: object list*)
 | *Durative-Plan-Action* (*name: name*) (*arguments: object list*) (*duration: time*)

type-synonym *plan* = (*time* × *plan-action*) *list*

1.2.5 Ground Actions

The following datatype represents an action scheme that has been instantiated by replacing the arguments with concrete objects, also called ground action.

datatype *ground-action* = *Ground-Action*
 (*precondition: (object atom) formula*)
 (*effect: object ast-effect*)

A happening represents a collection of grounded actions that are all executed simultaneously at a specified time point.

type-synonym *happening* = *time* × (*ground-action list*)

1.3 Closed-World Assumption, Equality, and Negation

Discriminator for atomic predicate formulas.

fun *is-predAtom* **where**
is-predAtom (*Atom* (*predAtm* - -)) = *True* | *is-predAtom* - = *False*

fun *is-eqAtom* **where**
is-eqAtom (*Atom* (*eqAtm* - -)) = *True* | *is-eqAtom* - = *False*

The world model is a set of (atomic) formulas

type-synonym *world-model* = *object atom formula set*

It is basic, if it only contains atoms

definition *wm-basic* $M \equiv \forall a \in M. \text{is-predAtom } a$

A valuation extracted from the atoms of the world model

definition *valuation* :: *world-model* $\Rightarrow$ *object atom valuation*
where *valuation* $M \equiv \lambda \text{predAtm } p \text{ xs} \Rightarrow \text{Atom } (\text{predAtm } p \text{ xs}) \in M \mid \text{eqAtm } a \text{ b} \Rightarrow a=b$

Augment a world model by adding negated versions of all atoms not contained in it, as well as interpretations of equality.

definition *close-world* :: *world-model* $\Rightarrow$ *world-model* **where**
close-world $M =$
 $M \cup \{\neg(\text{Atom } (\text{predAtm } p \text{ as})) \mid p \text{ as. } \text{Atom } (\text{predAtm } p \text{ as}) \notin M\}$
 $\cup \{\text{Atom } (\text{eqAtm } a \text{ a}) \mid a. \text{True}\} \cup \{\neg(\text{Atom } (\text{eqAtm } a \text{ b})) \mid a \text{ b. } a \neq b\}$

definition *close-neg* $M \equiv M \cup \{\neg(\text{Atom } a) \mid a. \text{Atom } a \notin M\}$

lemma *wm-basic* $M \implies \text{close-world } M = \text{close-neg } (M \cup \{\text{Atom } (\text{eqAtm } a \ a) \mid a. \text{ True}\})$
 $\langle \text{proof} \rangle$

abbreviation *cw-entailment* (**infix** $^c \models =$ 53) **where**
 $M \ ^c \models = \varphi \equiv \text{close-world } M \models \varphi$

lemma
close-world-extensive: $M \subseteq \text{close-world } M$ **and**
close-world-idem[simp]: $\text{close-world } (\text{close-world } M) = \text{close-world } M$
 $\langle \text{proof} \rangle$

lemma *in-close-world-conv*:
 $\varphi \in \text{close-world } M \longleftrightarrow ($
 $\varphi \in M$
 $\vee (\exists p \text{ as. } \varphi = \neg(\text{Atom } (\text{predAtm } p \ \text{as})) \wedge \text{Atom } (\text{predAtm } p \ \text{as}) \notin M)$
 $\vee (\exists a. \varphi = \text{Atom } (\text{eqAtm } a \ a))$
 $\vee (\exists a \ b. \varphi = \neg(\text{Atom } (\text{eqAtm } a \ b)) \wedge a \neq b)$
 $)$
 $\langle \text{proof} \rangle$

lemma *valuation-aux-1*:
fixes $M :: \text{world-model}$ **and** $\varphi :: \text{object atom formula}$
defines $C \equiv \text{close-world } M$
assumes $A: \forall \varphi \in C. \mathcal{A} \models \varphi$
shows $\mathcal{A} = \text{valuation } M$
 $\langle \text{proof} \rangle$

lemma *valuation-aux-2*:
assumes *wm-basic* M
shows $(\forall G \in \text{close-world } M. \text{valuation } M \models G)$
 $\langle \text{proof} \rangle$

lemma *val-imp-close-world*: $\text{valuation } M \models \varphi \implies M \ ^c \models = \varphi$
 $\langle \text{proof} \rangle$

lemma *close-world-imp-val*:
 $\text{wm-basic } M \implies M \ ^c \models = \varphi \implies \text{valuation } M \models \varphi$
 $\langle \text{proof} \rangle$

Main theorem of this section: If a world model M contains only atoms, its induced valuation satisfies a formula φ if and only if the closure of M entails φ .

Note that there are no syntactic restrictions on φ , in particular, φ may contain negation.

theorem *valuation-iff-close-world*:
assumes *wm-basic* M
shows $\text{valuation } M \models \varphi \longleftrightarrow M \ ^c \models = \varphi$
 $\langle \text{proof} \rangle$

1.3.1 Proper Generalization

Adding negation and equality is a proper generalization of the case without negation and equality

fun *is-STRIPS-fmla* :: 'ent atom formula $\Rightarrow$ bool **where**

```

  is-STRIPS-fmla (Atom (predAtm -))  $\longleftrightarrow$  True
| is-STRIPS-fmla ( $\perp$ )  $\longleftrightarrow$  True
| is-STRIPS-fmla ( $\varphi_1 \wedge \varphi_2$ )  $\longleftrightarrow$  is-STRIPS-fmla  $\varphi_1 \wedge$  is-STRIPS-fmla  $\varphi_2$ 
| is-STRIPS-fmla ( $\varphi_1 \vee \varphi_2$ )  $\longleftrightarrow$  is-STRIPS-fmla  $\varphi_1 \wedge$  is-STRIPS-fmla  $\varphi_2$ 
| is-STRIPS-fmla ( $\neg \perp$ )  $\longleftrightarrow$  True
| is-STRIPS-fmla -  $\longleftrightarrow$  False

```

lemma *aux1*: $\llbracket \text{wm-basic } M; \text{is-STRIPS-fmla } \varphi; \text{valuation } M \models \varphi; \forall G \in M. \mathcal{A} \models G \rrbracket \Rightarrow \mathcal{A} \models \varphi$
 <proof>

lemma *aux2*: $\llbracket \text{wm-basic } M; \text{is-STRIPS-fmla } \varphi; \forall \mathcal{A}. (\forall G \in M. \mathcal{A} \models G) \longrightarrow \mathcal{A} \models \varphi \rrbracket \Rightarrow \text{valuation } M \models \varphi$
 <proof>

lemma *valuation-iff-STRIPS*:

```

assumes wm-basic M
assumes is-STRIPS-fmla  $\varphi$ 
shows valuation M  $\models \varphi \longleftrightarrow M \models \varphi$ 
<proof>

```

Our extension to negation and equality is a proper generalization of the standard STRIPS semantics for formula without negation and equality

theorem *proper-STRIPS-generalization*:

```

 $\llbracket \text{wm-basic } M; \text{is-STRIPS-fmla } \varphi \rrbracket \Rightarrow M \models \varphi \longleftrightarrow M \models \varphi$ 
<proof>

```

1.4 Happening Execution Semantics

For this section, we fix a domain *D*, using Isabelle's locale mechanism.

```

locale ast-domain =
  fixes D :: ast-domain
begin

```

Check if two ground actions are interfering. This definition is taken from [FL03]

definition *acts-non-intrf* :: *ground-action* $\Rightarrow$ *ground-action* $\Rightarrow$ bool **where**
acts-non-intrf *a* *b* $\longleftrightarrow$ (
 let
 $\text{add}_a = \text{set}(\text{adds}(\text{effect } a)); \text{del}_a = \text{set}(\text{dels}(\text{effect } a)); \text{pre}_a = \text{Atom } \text{'atoms}$
 (*precondition* *a*);

$add_b = \text{set}(\text{adds}(\text{effect } b)); del_b = \text{set}(\text{dels}(\text{effect } b)); pre_b = \text{Atom } \text{'atoms}$
 (precondition b)
 in
 $pre_a \cap (add_b \cup del_b) = \{\}$ $\wedge$
 $pre_b \cap (add_a \cup del_a) = \{\}$ $\wedge$
 $add_a \cap del_b = \{\}$ $\wedge$
 $add_b \cap del_a = \{\}$

lemma *acts-non-intrf-symmetric*:

$\forall a \ b. \text{acts-non-intrf } a \ b \longleftrightarrow \text{acts-non-intrf } b \ a$
 <proof>

It seems to be agreed upon that, in case of a contradictory effect, addition overrides deletion. We model this behaviour by first executing the deletions, and then the additions.

fun *apply-happ* :: *happening* $\Rightarrow$ *world-model* $\Rightarrow$ *world-model* **where**
apply-happ (t_i, A_i) $M =$
 $(M - \bigcup (\text{set } (\text{map } (\text{set } o \text{dels } o \text{effect}) A_i))) \cup \bigcup (\text{set } (\text{map } (\text{set } o \text{adds } o \text{effect}) A_i))$

Predicate to model that the given list happenings is executable, and transforms an initial world model M into a final model M' .

Note that this definition over the list structure is more convenient in HOL than to explicitly define an indexed sequence $M_0 \dots M_N$ of intermediate world models, as done in [Lif87].

fun *valid-happ-seq* :: *world-model* $\Rightarrow$ *happening list* $\Rightarrow$ *world-model* $\Rightarrow$ *bool* **where**
valid-happ-seq $M \ \square \ M' \longleftrightarrow (M = M')$
 | *valid-happ-seq* $M \ ((t_i, A_i) \# \text{hs}) \ M' \longleftrightarrow$
 $(\forall a \in \text{set } A_i. M \stackrel{c}{\models} \text{precondition } a)$
 $\wedge (\text{pairwise acts-non-intrf } (\text{set } A_i))$
 $\wedge \text{valid-happ-seq } (\text{apply-happ } (t_i, A_i) \ M) \ \text{hs} \ M'$

lemma *valid-happ-seq- M_j -is- M_i* :

assumes $\forall h \in \text{set } \text{hs}. \text{apply-happ } h \ M_i = M_i$

and *valid-happ-seq* $M_i \ \text{hs} \ M_j$

shows $M_i = M_j$

<proof>

end — Context of *ast-domain*

1.5 Well-Formedness of PDDL

fun *ty-term* **where**

ty-term *varT objT* (*term*.*VAR* v) = *varT* v

| *ty-term* *varT objT* (*term*.*CONST* c) = *objT* c

lemma *ty-term-mono*: $\text{varT} \subseteq_m \text{varT}' \Longrightarrow \text{objT} \subseteq_m \text{objT}' \Longrightarrow$

ty-term *varT objT* $\subseteq_m$ *ty-term* *varT' objT'*
 ⟨*proof*⟩

context *ast-domain* **begin**

The signature is a partial function that maps the predicates of the domain to lists of argument types.

definition *sig* :: *predicate* $\rightarrow$ *type list* **where**
sig $\equiv$ *map-of* (*map* (λ *PredDecl* *p n* $\Rightarrow$ (*p,n*)) (*predicates D*))

definition *func-sig* :: *func* $\rightarrow$ *type list* **where**
func-sig $\equiv$ *map-of* (*map* (λ *FuncDecl* *f n* $\Rightarrow$ (*f,n*)) (*functions D*))

We use a flat subtype hierarchy, where every type is a subtype of object, and there are no other subtype relations.

Note that we do not need to restrict this relation to declared types, as we will explicitly ensure that all types used in the problem are declared.

fun *subtype-edge* **where**
subtype-edge (*ty,superty*) = (*superty,ty*)

definition *subtype-rel* $\equiv$ *set* (*map* *subtype-edge* (*types D*))

definition *of-type* :: *type* $\Rightarrow$ *type* $\Rightarrow$ *bool* **where**
of-type *oT T* $\equiv$ *set* (*primitives oT*) $\subseteq$ *subtype-rel** “ *set* (*primitives T*)

This checks that every primitive on the LHS is contained in or a subtype of a primitive on the RHS

For the next few definitions, we fix a partial function that maps a polymorphic entity type '*e*' to types. An entity can be instantiated by variables or objects later.

context
fixes *ty-ent* :: '*ent* $\rightarrow$ *type* — Entity's type, None if invalid
begin

Checks whether an entity has a given type

definition *is-of-type* :: '*ent* $\Rightarrow$ *type* $\Rightarrow$ *bool* **where**
is-of-type *v T* $\longleftrightarrow$ (
 case *ty-ent v* *of*
 Some vT $\Rightarrow$ *of-type vT T*
 | *None* $\Rightarrow$ *False*)

fun *wf-pred-atom* :: *predicate* $\times$ '*ent list* $\Rightarrow$ *bool* **where**
wf-pred-atom (*p,vs*) $\longleftrightarrow$ (
 case *sig p* *of*
 None $\Rightarrow$ *False*

```

| Some Ts  $\Rightarrow$  list-all2 is-of-type vs Ts)

fun wf-func-args :: func  $\times$  'ent list  $\Rightarrow$  bool where
  wf-func-args (f,vs)  $\longleftrightarrow$  (
    case func-sig f of
      None  $\Rightarrow$  False
    | Some Ts  $\Rightarrow$  list-all2 is-of-type vs Ts)

```

Predicate-atoms are well-formed if their arguments match the signature, equalities are well-formed if the arguments are valid objects (have a type).

TODO: We could check that types may actually overlap

```

fun wf-atom :: 'ent atom  $\Rightarrow$  bool where
  wf-atom (predAtm p vs)  $\longleftrightarrow$  wf-pred-atom (p,vs)
| wf-atom (eqAtm a b)  $\longleftrightarrow$  ty-ent a  $\neq$  None  $\wedge$  ty-ent b  $\neq$  None

```

A formula is well-formed if it consists of valid atoms, and does not contain negations, except for the encoding $\neg \perp$ of true.

```

fun wf-fmla :: ('ent atom) formula  $\Rightarrow$  bool where
  wf-fmla (Atom a)  $\longleftrightarrow$  wf-atom a
| wf-fmla ( $\perp$ )  $\longleftrightarrow$  True
| wf-fmla ( $\varphi 1 \wedge \varphi 2$ )  $\longleftrightarrow$  (wf-fmla  $\varphi 1 \wedge$  wf-fmla  $\varphi 2$ )
| wf-fmla ( $\varphi 1 \vee \varphi 2$ )  $\longleftrightarrow$  (wf-fmla  $\varphi 1 \wedge$  wf-fmla  $\varphi 2$ )
| wf-fmla ( $\neg \varphi$ )  $\longleftrightarrow$  wf-fmla  $\varphi$ 
| wf-fmla ( $\varphi 1 \rightarrow \varphi 2$ )  $\longleftrightarrow$  (wf-fmla  $\varphi 1 \wedge$  wf-fmla  $\varphi 2$ )

```

```

lemma wf-fmla  $\varphi = (\forall a \in \text{atoms } \varphi. \text{wf-atom } a)$ 
  <proof>

```

Special case for a well-formed atomic predicate formula

```

fun wf-fmla-atom where
  wf-fmla-atom (Atom (predAtm a vs))  $\longleftrightarrow$  wf-pred-atom (a,vs)
| wf-fmla-atom -  $\longleftrightarrow$  False

```

```

lemma wf-fmla-atom-alt: wf-fmla-atom  $\varphi \longleftrightarrow \text{is-predAtom } \varphi \wedge \text{wf-fmla } \varphi$ 
  <proof>

```

An effect is well-formed if the added and removed formulas are atomic

```

fun wf-effect where
  wf-effect (Effect a d)  $\longleftrightarrow (\forall ae \in \text{set } a. \text{wf-fmla-atom } ae) \wedge (\forall de \in \text{set } d. \text{wf-fmla-atom } de)$ 

```

```

fun wf-duration-const :: 'ent duration-constraint  $\Rightarrow$  bool where
  wf-duration-const No-Const  $\longleftrightarrow$  True
| wf-duration-const (Time-Const op d)  $\longleftrightarrow d \geq 0$ 
| wf-duration-const (Func-Const op f vs)  $\longleftrightarrow$ 
  (case func-sig f of
    None  $\Rightarrow$  False
  | Some Ts  $\Rightarrow$  list-all2 is-of-type vs Ts)

```

end — Context fixing *ty-ent*

definition $constT :: object \rightarrow type$ **where**
 $constT \equiv map-of (consts D)$

An action schema is well-formed if the parameter names are distinct, and the precondition and effect is well-formed wrt. the parameters.

fun *wf-action-schema* :: *ast-action-schema* $\Rightarrow$ *bool* **where**
 $wf-action-schema (Simple-Action-Schema\ n\ params\ pre\ eff) \longleftrightarrow ($
 $\quad let\ tyt = ty-term\ (map-of\ params)\ constT\ in$
 $\quad distinct\ (map\ fst\ params)$
 $\quad \wedge\ wf-fmla\ tyt\ pre$
 $\quad \wedge\ wf-effect\ tyt\ eff)$
 $| wf-action-schema (Durative-Action-Schema\ n\ params\ d\ cond\ eff) \longleftrightarrow ($
 $\quad let\ tyt = ty-term\ (map-of\ params)\ constT\ in$
 $\quad distinct\ (map\ fst\ params)$
 $\quad \wedge\ (\forall (t,c) \in set\ cond.\ wf-fmla\ tyt\ c)$
 $\quad \wedge\ (\forall (t,e) \in set\ eff.\ wf-effect\ tyt\ e \wedge t \neq Over-All)$
 $\quad \wedge\ wf-duration-const\ tyt\ d)$

A type is well-formed if it consists only of declared primitive types, and the type object.

fun *wf-type* **where**
 $wf-type (Either\ Ts) \longleftrightarrow set\ Ts \subseteq insert\ "object"\ (fst'set\ (types\ D))$

A predicate is well-formed if its argument types are well-formed.

fun *wf-predicate-decl* **where**
 $wf-predicate-decl (PredDecl\ p\ Ts) \longleftrightarrow (\forall T \in set\ Ts.\ wf-type\ T)$

A function is well-formed if its argument types are well-formed.

fun *wf-function-decl* **where**
 $wf-function-decl (FuncDecl\ f\ Ts) \longleftrightarrow (\forall T \in set\ Ts.\ wf-type\ T)$

The types declaration is well-formed, if all supertypes are declared types (or object)

definition $wf-types \equiv snd'set\ (types\ D) \subseteq insert\ "object"\ (fst'set\ (types\ D))$

A domain is well-formed if

- there are no duplicate declared predicate names,
- all declared predicates are well-formed,
- there are no duplicate declared function names,
- all declared functions are well-formed,

- there are no duplicate action names,
- and all declared actions are well-formed

definition *wf-domain* :: *bool* **where**

wf-domain $\equiv$
wf-types
 $\wedge$ *distinct* (*map* (*predicate-decl.pred*) (*predicates D*))
 $\wedge$ ($\forall p \in \text{set } (\text{predicates } D). \text{wf-predicate-decl } p$)
 $\wedge$ *distinct* (*map* (*function-decl.func*) (*functions D*))
 $\wedge$ ($\forall f \in \text{set } (\text{functions } D). \text{wf-function-decl } f$)
 $\wedge$ *distinct* (*map fst* (*consts D*))
 $\wedge$ ($\forall (n, T) \in \text{set } (\text{consts } D). \text{wf-type } T$)
 $\wedge$ *distinct* (*map ast-action-schema.name* (*actions D*))
 $\wedge$ ($\forall a \in \text{set } (\text{actions } D). \text{wf-action-schema } a$)

end — locale *ast-domain*

We fix a problem, and also include the definitions for the domain of this problem.

locale *ast-problem* = *ast-domain domain P*
for *P* :: *ast-problem*
begin

We refer to the problem domain as *D*

abbreviation *D* $\equiv$ *ast-problem.domain P*

definition *objT* :: *object* $\rightarrow$ *type* **where**

objT $\equiv$ *map-of* (*objects P*) ++ *constT*

lemma *objT-alt*: *objT* = *map-of* (*consts D* @ *objects P*)
 $\langle \text{proof} \rangle$

definition *wf-fact* :: *fact* $\Rightarrow$ *bool* **where**

wf-fact = *wf-pred-atom objT*

fun *wf-func-assign* :: *object atom formula* $\Rightarrow$ *bool* **where**

wf-func-assign (*Atom* (*eqAtm* (*FuncEnt f args*) (*TimeEnt d*))) $\longleftrightarrow$ *wf-func-args*
objT (*f, args*)
| *wf-func-assign* - $\longleftrightarrow$ *False*

lemma *wf-func-assign-is-eqAtom*: *wf-func-assign f* $\implies$ *is-eqAtom f*
 $\langle \text{proof} \rangle$

This definition is needed for well-formedness of the initial model, and forward-references to the concept of world model.

definition *wf-world-model* **where**

wf-world-model $M = (\forall f \in M. \text{wf-fmla-atom } \text{objT } f)$

definition *wf-problem* **where**

wf-problem $\equiv$
wf-domain
 $\wedge \text{distinct } (\text{map fst } (\text{objects } P) \text{ @ } \text{map fst } (\text{consts } D))$
 $\wedge (\forall (n, T) \in \text{set } (\text{objects } P). \text{wf-type } T)$
 $\wedge \text{distinct } (\text{init } P)$
 $\wedge (\forall f \in \text{set } (\text{init } P). \text{wf-fmla-atom } \text{objT } f \vee \text{wf-func-assign } f)$
 $\wedge \text{wf-fmla } \text{objT } (\text{goal } P)$

fun *wf-effect-inst* :: *object ast-effect* $\Rightarrow$ *bool* **where**

wf-effect-inst (*Effect* (*a*) (*d*)) $\longleftrightarrow (\forall a \in \text{set } a \cup \text{set } d. \text{wf-fmla-atom } \text{objT } a)$

lemma *wf-effect-inst-alt*: *wf-effect-inst eff* = *wf-effect objT eff*

<proof>

end — locale *ast-problem*

Locale to express a well-formed domain

locale *wf-ast-domain* = *ast-domain* +
assumes *wf-domain*: *wf-domain*

Locale to express a well-formed problem

locale *wf-ast-problem* = *ast-problem* *P* **for** *P* +
assumes *wf-problem*: *wf-problem*

begin

sublocale *wf-ast-domain domain P*
<proof>

end — locale *wf-ast-problem*

1.6 PDDL Semantics

context *ast-domain* **begin**

definition *resolve-action-schema* :: *name* $\rightarrow$ *ast-action-schema* **where**

resolve-action-schema *n* = *index-by ast-action-schema.name (actions D) n*

fun *subst-term* **where**

subst-term psubst (term.VAR x) = *psubst x*
| *subst-term psubst (term.CONST c)* = *c*

To instantiate an action schema, we first compute a substitution from parameters to objects, and then apply this substitution to the precondition and effect. The substitution is applied via the *map-xxx* functions generated by the datatype package.

```

fun instantiate-action-schema :: ast-action-schema  $\Rightarrow$  object list  $\Rightarrow$  ground-action
where
  instantiate-action-schema (Simple-Action-Schema n params pre eff) args =
    (let
      tsubst = subst-term (the o (map-of (zip (map fst params) args)));
      pre-inst = (map-formula o map-atom) tsubst pre;
      eff-inst = (map-ast-effect) tsubst eff
    in
      Ground-Action pre-inst eff-inst
    )

```

Filter a given timed-list by a time specifier. (**at start** ...), (**at end** ...), or (**over all** ...) conditions/effects

```

definition filter-time-spec :: temporal-annotation  $\Rightarrow$  (temporal-annotation  $\times$  'a)
list  $\Rightarrow$  'a list where
  filter-time-spec t l = map snd (filter (((=) t) o fst) l)

```

```

lemma filter-time-spec-correct:  $\forall x \in \text{set} \text{ (filter-time-spec } t \text{ } xs).$   $(t, x) \in \text{set } xs$ 
<proof>

```

auxiliary-function to conjunct multiple effects into one single effect. Needed to construct effect after filtering by a time specifier

```

definition conjunct-effects :: ('a ast-effect) list  $\Rightarrow$  'a ast-effect ( $\bigwedge_{eff} - [40] \ 40$ )
where
  conjunct-effects l = Effect ((concat o (map adds)) l) ((concat o (map dels)) l)

```

instantiate a given Durative Action Schema; Note: if an action schema is well formed then the effect for the snap action for the temporal annotation (**over all** ...) will be an empty effect (**Effect** [] [])

```

fun inst-snap-action :: ast-action-schema  $\Rightarrow$  object list  $\Rightarrow$  temporal-annotation
 $\Rightarrow$  ground-action where
  inst-snap-action (Durative-Action-Schema n params d cond eff) args ta =
    (let
      tsubst = subst-term (the o (map-of (zip (map fst params) args)));
      prei = (map-formula o map-atom) tsubst (BigAnd (filter-time-spec ta cond));
      effi = (map-ast-effect) tsubst ( $\bigwedge_{eff}$  (filter-time-spec ta eff))
    in
      Ground-Action prei effi
    )

```

end — Context of *ast-domain*

context wf-ast-domain **begin**

Resolving an action yields a well-founded action schema.

```

lemma resolve-action-wf:
  assumes resolve-action-schema n = Some a
  shows wf-action-schema a

```

$\langle proof \rangle$

end — Context of *ast-domain*

context *ast-problem* **begin**

Initial model

definition *I* :: *world-model* **where**
I $\equiv$ *set* (*filter is-predAtom* (*init P*))

Resolve a plan action and instantiate the referenced action schema.

fun *res-inst* :: *plan-action* $\rightarrow$ *ground-action* **where**
res-inst (*Simple-Plan-Action* *n* *args*) =
Some (*instantiate-action-schema* (*the* (*resolve-action-schema* *n*)) *args*)
| *res-inst* (*Durative-Plan-Action* - -) = *None*

fun *res-inst-snap-action* :: *plan-action* $\Rightarrow$ *temporal-annotation* $\rightarrow$ *ground-action*
where
res-inst-snap-action (*Durative-Plan-Action* *n* *args* *d*) *ta* =
Some (*inst-snap-action* (*the* (*resolve-action-schema* *n*)) *args* *ta*)
| *res-inst-snap-action* (*Simple-Plan-Action* - -) = *None*

Check whether object has specified type

definition *is-obj-of-type* *n* *T* $\equiv$ *case objT* *n* *of*
None $\Rightarrow$ *False*
| *Some* *oT* $\Rightarrow$ *of-type* *oT* *T*

We can also use the generic *is-of-type* function.

lemma *is-obj-of-type-alt*: *is-obj-of-type* = *is-of-type* *objT*
 $\langle proof \rangle$

HOL encoding of matching an action's formal parameters against an argument list. The parameters of the action are encoded as a list of *name* $\times$ *type* pairs, such that we map it to a list of types first. Then, the list relator *list-all2* checks that arguments and types have the same length, and each matching pair of argument and type satisfies the predicate *is-obj-of-type*.

definition *action-params-match* *a* *args*
 $\equiv$ *list-all2 is-obj-of-type* *args* (*map snd* (*parameters* *a*))

All functions are required to be constant and defined in the initial state. Therefore all duration constraints must be satisfied by the initial state.

definition *func-eval* :: *object* $\rightarrow$ *rat* **where**
func-eval $\equiv$ *map-of* (*fold* (λa *fev*. *case* *a* *of* *Atom* (*eqAtm* *f* (*TimeEnt* *t*)) $\Rightarrow$ (*f,t*)#*fev* | - $\Rightarrow$ *fev*) (*init P*) [])

fun *duration-matches* :: *time* $\Rightarrow$ *term* *duration-constraint* $\Rightarrow$ (*variable* $\times$ *type*)
list $\Rightarrow$ *object* *list* $\Rightarrow$ *bool* **where**


```

duration-matches d No-Const params args  $\longleftrightarrow d \geq 0$ 
| duration-matches d (Time-Const op d') params args  $\longleftrightarrow$ 
  (case op of
    LEQ  $\Rightarrow d \leq d'$ 
  | EQ  $\Rightarrow d = d'$ 
  | GEQ  $\Rightarrow d \geq d'$ )
| duration-matches d (Func-Const op f vs) params args  $\longleftrightarrow$ 
  (let tsubst = subst-term (the o (map-of (zip (map fst params) args))) in
    case func-eval (FuncEnt f (map tsubst vs)) of
      None  $\Rightarrow$  False
    | Some d'  $\Rightarrow$  (case op of
      LEQ  $\Rightarrow d \leq d'$ 
    | EQ  $\Rightarrow d = d'$ 
    | GEQ  $\Rightarrow d \geq d'$ ))

```

At this point, we can define well-formedness of a plan action: The action must refer to a declared action schema, the arguments must be compatible with the formal parameters' types.

```

fun wf-plan-action :: plan-action  $\Rightarrow$  bool where
  wf-plan-action (Simple-Plan-Action n args) = (
    case resolve-action-schema n of
      None  $\Rightarrow$  False
    | Some (Simple-Action-Schema n params pre eff)  $\Rightarrow$ 
      action-params-match (Simple-Action-Schema n params pre eff) args
    | Some (Durative-Action-Schema n params d cond eff)  $\Rightarrow$  False)
  | wf-plan-action (Durative-Plan-Action n args d) = (
    case resolve-action-schema n of
      None  $\Rightarrow$  False
    | Some (Simple-Action-Schema n params pre eff)  $\Rightarrow$  False
    | Some (Durative-Action-Schema n params dconst cond eff)  $\Rightarrow$ 
      action-params-match (Durative-Action-Schema n params dconst cond eff)
args
     $\wedge$  duration-matches d dconst params args  $\wedge d \geq 0$ )

```

A plan is wellformed if all plan actions are wellformed and all starting time points are greater-equal 0.

definition wf-plan :: plan $\Rightarrow$ bool **where**
 wf-plan $\pi s \longleftrightarrow (\forall (t, \pi) \in \text{set } \pi s. \text{wf-plan-action } \pi \wedge t \geq 0)$

definition is-act-simple :: plan-action $\Rightarrow$ bool **where**
 is-act-simple $\pi \longleftrightarrow (\text{case } \pi \text{ of } (\text{Simple-Plan-Action } -) \Rightarrow \text{True} \mid - \Rightarrow \text{False})$

definition simple-acts :: plan $\Rightarrow$ (time $\times$ plan-action) set **where**
 simple-acts $\pi s = \text{set } (\text{filter } (\text{is-act-simple } o \text{snd}) \pi s)$

lemma simple-acts-subset: simple-acts $\pi s \subseteq \text{set } \pi s$
 <proof>

definition durative-acts :: plan $\Rightarrow$ (time $\times$ plan-action) set **where**

$durative-acts \pi s = set (filter ((HOL.Not) o is-act-simple o snd) \pi s)$

lemma *dur-acts-subset*: $durative-acts \pi s \subseteq set \pi s$
 $\langle proof \rangle$

Definition of a happening time point. Happening time point specify time points at, which the current state can change. Therefore every start and end of any plan action is a happening time point.

definition *is-htp* :: $plan \Rightarrow time \Rightarrow bool$ **where**
 $is-htp \pi s t_i \longleftrightarrow (\exists \pi. (t_i, \pi) \in set \pi s) \vee (\exists (t_\pi, \pi) \in durative-acts \pi s. t_i = t_\pi + (duration \pi))$

Predicate for sequence of happening time points.

definition *htps-seq* :: $plan \Rightarrow time list \Rightarrow bool$ **where**
 $htps-seq \pi s htps \longleftrightarrow strict-sorted htps \wedge (\forall t. t \in set htps \longleftrightarrow is-htp \pi s t)$

Predicate for consecutive happening time points: There is no happening time point in between.

definition *consec-htps* :: $plan \Rightarrow time \Rightarrow time \Rightarrow bool$ **where**
 $consec-htps \pi s t_i t_j \longleftrightarrow (t_i < t_j \wedge is-htp \pi s t_i \wedge is-htp \pi s t_j \wedge (\forall t. is-htp \pi s t \longrightarrow t \leq t_i \vee t_j \leq t))$

Predicate to check is a grounded action is a instantiation of a plan action for an induced happening sequence.

fun *inst-of-plan-action* :: $plan \Rightarrow time \times plan-action \Rightarrow time \times ground-action \Rightarrow bool$ **where**
 $inst-of-plan-action \pi s (t_\pi, \pi) (t_a, a) \longleftrightarrow ($
 $(res-inst \pi = Some a \wedge t_\pi = t_a)$
 $\vee (res-inst-snap-action \pi At-Start = Some a \wedge t_\pi = t_a)$
 $\vee (res-inst-snap-action \pi At-End = Some a \wedge t_\pi + (duration \pi) = t_a)$
 $\vee (res-inst-snap-action \pi Over-All = Some a \wedge$
 $(\exists t_i t_j. consec-htps \pi s t_i t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (duration \pi) \wedge t_i < t_a$
 $\wedge t_a < t_j)))$

abbreviation *happ-member* (**infix** $\in_h$ 53) **where**
 $ta \in_h hs \equiv (case ta of (t, a) \Rightarrow (\exists as. (t, as) \in set hs \wedge a \in set as))$

Definition of an induced happening sequence.

- for every simple action (t, π) there is an instantiation at time point t
- for every durative action (t, π)
 - there is a ground action at time point t , which contains all conditions and effect of π with time specifier (**at start ...**)

- there is a ground action at time point $t + (\text{duration } \pi)$, which contains all conditions and effect of π with time specifier (**at end ...**)
- in between every consecutive happening-time-points t_i t_j during the execution of π there is a ground action at some time point t' (in between t_i & t_j), which contains all conditions of π with time specifier (**over all ...**), the effect of is empty
- additionally an induced happening sequence is strictly sorted and contains no other actions besides the ones specified above.

definition *ind-happ-seq* :: *plan* $\Rightarrow$ *happening list* $\Rightarrow$ *bool* **where**

ind-happ-seq πs $hs \longleftrightarrow$ (
strict-sorted (*map fst* hs)
 $\wedge (\forall (t_\pi, \pi) \in \text{simple-acts } \pi s.$
 let $g_a = \text{the } (\text{res-inst } \pi) \text{ in}$
 $\exists A. (t_\pi, A) \in \text{set } hs \wedge g_a \in \text{set } A)$
 $\wedge (\forall (t_\pi, \pi) \in \text{durative-acts } \pi s.$
 let $\pi_{start} = \text{the } (\text{res-inst-snap-action } \pi \text{ At-Start});$
 $\pi_{end} = \text{the } (\text{res-inst-snap-action } \pi \text{ At-End});$
 $\pi_{inv} = \text{the } (\text{res-inst-snap-action } \pi \text{ Over-All}) \text{ in}$
 $(\exists A. (t_\pi, A) \in \text{set } hs \wedge \pi_{start} \in \text{set } A) \wedge$
 $(\exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge \pi_{end} \in \text{set } A) \wedge$
 $(\forall t_i t_j. (\text{consec-htps } \pi s t_i t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (\text{duration } \pi))$
 $\longrightarrow (\exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge \pi_{inv} \in \text{set } A)))$
 $\wedge (\forall (t_i, A_i) \in \text{set } hs. A_i \neq [] \wedge$
 $(\forall a \in \text{set } A_i. \exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_i, a))))$

lemma *ind-happ-seq-props*:

assumes *ind-happ-seq* πs hs

shows *strict-sorted* (*map fst* hs)

$(t_\pi, \pi) \in \text{simple-acts } \pi s \implies \exists A. (t_\pi, A) \in \text{set } hs \wedge \text{the } (\text{res-inst } \pi) \in \text{set } A$

$(t_\pi, \pi) \in \text{durative-acts } \pi s \implies \exists A. (t_\pi, A) \in \text{set } hs \wedge \text{the } (\text{res-inst-snap-action } \pi \text{ At-Start}) \in \text{set } A$

$(t_\pi, \pi) \in \text{durative-acts } \pi s \implies \exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge \text{the } (\text{res-inst-snap-action } \pi \text{ At-End}) \in \text{set } A$

$\llbracket (t_\pi, \pi) \in \text{durative-acts } \pi s; \text{consec-htps } \pi s t_i t_j ; t_\pi \leq t_i ; t_j \leq t_\pi + (\text{duration } \pi) \rrbracket$

$\implies \exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge \text{the } (\text{res-inst-snap-action } \pi \text{ Over-All}) \in \text{set } A$

$(t_i, A_i) \in \text{set } hs \implies A_i \neq []$

$\llbracket (t_i, A_i) \in \text{set } hs ; a \in \text{set } A_i \rrbracket \implies \exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_i, a)$

<proof>

A sequence of plan actions form a path, if they are well-formed and their instantiations form a path.

definition *plan-happ-path* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *world-model* $\Rightarrow$ *bool* **where**

$plan-happ-path\ M\ \pi s\ M' \longleftrightarrow (\exists\ hs.\ ind-happ-seq\ \pi s\ hs \wedge valid-happ-seq\ M\ hs\ M')$

A plan is valid wrt. a given initial model, if it forms a path to a goal model

definition $valid-plan-from2 :: world-model \Rightarrow plan \Rightarrow bool$ **where**
 $valid-plan-from2\ M\ \pi s \longleftrightarrow (wf-plan\ \pi s \wedge (\exists\ M'. plan-happ-path\ M\ \pi s\ M' \wedge M' \models (goal\ P)))$

Finally, a plan is valid if it is valid wrt. the initial world model I

definition $valid-plan2 :: plan \Rightarrow bool$
where $valid-plan2 \equiv valid-plan-from2\ I$

Concise definition used in paper:

lemma $valid-plan2\ \pi s \equiv wf-plan\ \pi s \wedge (\exists\ M'. plan-happ-path\ I\ \pi s\ M' \wedge M' \models (goal\ P))$
 $\langle proof \rangle$

end — Context of *ast-problem*

1.7 Preservation of Well-Formedness

1.7.1 Well-Formed Action Instances

The goal of this section is to establish that well-formedness of world models is preserved by execution of well-formed plan actions.

context *ast-problem* **begin**

As plan actions are executed by first instantiating them, and then executing the action instance, it is natural to define a well-formedness concept for action instances.

fun $wf-ground-action :: ground-action \Rightarrow bool$ **where**
 $wf-ground-action\ (Ground-Action\ pre\ eff) \longleftrightarrow (wf-fmla\ objT\ pre \wedge wf-effect\ objT\ eff)$

We first prove that instantiating a well-formed action schema will yield a well-formed action instance.

We begin with some auxiliary lemmas before the actual theorem.

lemma (**in** *ast-domain*) $of-type-refl[simp, intro!]: of-type\ T\ T$
 $\langle proof \rangle$

lemma (**in** *ast-domain*) $of-type-trans[trans]:$
 $of-type\ T1\ T2 \implies of-type\ T2\ T3 \implies of-type\ T1\ T3$
 $\langle proof \rangle$

lemma $is-of-type-map-ofE:$
assumes $is-of-type\ (map-of\ params)\ x\ T$
obtains $i\ xT$ **where** $i < length\ params$ $params!i = (x, xT)$ $of-type\ xT\ T$

$\langle proof \rangle$

lemma *wf-atom-mono*:

assumes *SS*: $tys \subseteq_m tys'$

assumes *WF*: *wf-atom* *tys* *a*

shows *wf-atom* *tys'* *a*

$\langle proof \rangle$

lemma *wf-fmla-atom-mono*:

assumes *SS*: $tys \subseteq_m tys'$

assumes *WF*: *wf-fmla-atom* *tys* *a*

shows *wf-fmla-atom* *tys'* *a*

$\langle proof \rangle$

lemma *constT-ss-objT*: $constT \subseteq_m objT$

$\langle proof \rangle$

lemma *wf-atom-constT-imp-objT*: *wf-atom* (*ty-term* *Q* *constT*) *a* $\implies$ *wf-atom* (*ty-term* *Q* *objT*) *a*

$\langle proof \rangle$

lemma *wf-fmla-atom-constT-imp-objT*: *wf-fmla-atom* (*ty-term* *Q* *constT*) *a* $\implies$ *wf-fmla-atom* (*ty-term* *Q* *objT*) *a*

$\langle proof \rangle$

context

fixes *Q* and *f* :: *variable* $\Rightarrow$ *object*

assumes *INST*: *is-of-type* *Q* *x* *T* $\implies$ *is-of-type* *objT* (*f* *x*) *T*

begin

lemma *is-of-type-var-conv*: *is-of-type* (*ty-term* *Q* *objT*) (*term.VAR* *x*) *T* $\longleftrightarrow$ *is-of-type* *Q* *x* *T*

$\langle proof \rangle$

lemma *is-of-type-const-conv*: *is-of-type* (*ty-term* *Q* *objT*) (*term.CONST* *x*) *T* $\longleftrightarrow$ *is-of-type* *objT* *x* *T*

$\langle proof \rangle$

lemma *INST'*: *is-of-type* (*ty-term* *Q* *objT*) *x* *T* $\implies$ *is-of-type* *objT* (*subst-term* *f* *x*) *T*

$\langle proof \rangle$

lemma *wf-inst-eq-aux*: $Q\ x = \text{Some } T \implies objT\ (f\ x) \neq \text{None}$

$\langle proof \rangle$

lemma *wf-inst-eq-aux'*: *ty-term* *Q* *objT* *x* = *Some* *T* $\implies$ *objT* (*subst-term* *f* *x*) $\neq \text{None}$

$\langle proof \rangle$

lemma *wf-inst-atom*:

assumes *wf-atom* (*ty-term* *Q* *constT*) *a*

shows *wf-atom* *objT* (*map-atom* (*subst-term* *f*) *a*)

$\langle proof \rangle$

lemma *wf-inst-formula-atom*:

assumes *wf-fmla-atom* (*ty-term* *Q* *constT*) *a*

shows *wf-fmla-atom* *objT* ((*map-formula* *o* *map-atom* *o* *subst-term*) *f* *a*)

$\langle proof \rangle$

lemma *wf-inst-effect*:

assumes *wf-effect* (*ty-term* *Q* *constT*) φ

shows *wf-effect* *objT* ((*map-ast-effect* *o* *subst-term*) *f* φ)

$\langle proof \rangle$

lemma *wf-inst-formula*:

assumes *wf-fmla* (*ty-term* *Q* *constT*) φ

shows *wf-fmla* *objT* ((*map-formula* *o* *map-atom* *o* *subst-term*) *f* φ)

$\langle proof \rangle$

lemma *wf-BigAnd-aux*: $\forall x \in \text{set } xs. \text{wf-fmla } tyt \ x \implies \text{wf-fmla } tyt \ (\text{BigAnd } xs)$

$\langle proof \rangle$

lemma *wf-BigAnd-filter-time-spec*:

fixes *n* *params* *d* *cond* *eff*

defines *a* $\equiv$ *Durative-Action-Schema* *n* *params* *d* *cond* *eff*

assumes *wf-action-schema* *a*

shows *wf-fmla* (*ty-term* (*map-of* *params*) *constT*) (*BigAnd* (*filter-time-spec* *ta* *cond*))

$\langle proof \rangle$

lemma *wf-conjunct-effects-aux*: $\forall x \in \text{set } xs. \text{wf-effect } tyt \ x \implies \text{wf-effect } tyt \ (\bigwedge_{eff} xs)$

$\langle proof \rangle$

lemma *wf-conj-eff-filter-time-spec*:

fixes *n* *params* *d* *cond* *eff*

defines *a* $\equiv$ *Durative-Action-Schema* *n* *params* *d* *cond* *eff*

assumes *wf-action-schema* *a*

shows *wf-effect* (*ty-term* (*map-of* *params*) *constT*) ($\bigwedge_{eff} (\text{filter-time-spec } ta \text{ } eff)$)

$\langle proof \rangle$

end

Next two theorems show that, instantiating a well-formed action schema

with compatible arguments will yield a well-formed action instance.

theorem *wf-inst-action-schema*:
fixes n *params* *pre* *eff*
defines $a \equiv \text{Simple-Action-Schema } n \text{ params pre eff}$
assumes *action-params-match* a *args*
assumes *wf-action-schema* a
shows *wf-ground-action* (*instantiate-action-schema* a *args*)
 $\langle \text{proof} \rangle$

theorem *wf-inst-durative-action-schema*:
fixes n *params* d *cond* *eff*
defines $a \equiv \text{Durative-Action-Schema } n \text{ params } d \text{ cond eff}$
assumes *action-params-match* a *args*
assumes *wf-action-schema* a
shows *wf-ground-action* (*inst-snap-action* a *args* ta)
 $\langle \text{proof} \rangle$

lemma *wf-over-all-empty-eff*:
assumes *wf-plan-action* π
and *wf-action-schema* a_{schema}
and *Some* $a_{\text{schema}} = \text{resolve-action-schema (name } \pi)$
and *Some* $a = \text{res-inst-snap-action } \pi \text{ Over-All}$
shows *effect* $a = \text{Effect } [] []$
 $\langle \text{proof} \rangle$

end — Context of *ast-problem*

1.7.2 Preservation

context *ast-problem* **begin**

fun *happ-precond* :: *happening* $\Rightarrow$ *world-model* $\Rightarrow$ *bool* **where**
happ-precond (t, A) $M \longleftrightarrow (\forall a \in \text{set } A. M \models_{\text{c}} \text{precondition } a)$

fun *happ-non-intrf* :: *happening* $\Rightarrow$ *bool* **where**
happ-non-intrf (t, A) $\longleftrightarrow (\forall a \in \text{set } A. \forall b \in \text{set } A. a \neq b \longrightarrow \text{acts-non-intrf } a \text{ } b)$

Shorthand for enabled happening: all preconditions satisfied, no interfering actions, & all ground actions are wellformed .

fun *happ-enabled* :: *happening* $\Rightarrow$ *world-model* $\Rightarrow$ *bool* **where**
happ-enabled (t, A) $M \longleftrightarrow (\text{happ-precond } (t, A) \text{ } M \wedge \text{happ-non-intrf } (t, A) \wedge (\forall a \in \text{set } A. \text{wf-ground-action } a))$

Shorthand for wellformed-ness of a happening sequence: all ground actions wellformed .

fun *wf-happ-seq* :: *happening list* $\Rightarrow$ *bool* **where**
wf-happ-seq $hs \longleftrightarrow (\forall (t, A) \in \text{set } hs. (\forall a \in \text{set } A. \text{wf-ground-action } a))$

end — Context of *ast-problem*

context *wf-ast-problem* **begin**

The initial world model is well-formed

lemma *wf-I: wf-world-model I*
⟨*proof*⟩

lemma *wf-ground-action-wf-fmla-atom:*
 $wf-ground-action\ a \implies \forall x \in set\ ((adds\ o\ effect)\ a). wf-fmla-atom\ objT\ x$
⟨*proof*⟩

The Application of a happening preserves well-formedness

theorem *wf-apply-happ:*
assumes *happ-enabled (t,as) s*
and *wf-world-model s*
shows *wf-world-model (apply-happ (t,as) s)*
⟨*proof*⟩

theorem *wf-apply-happ-compact-notation:*
 $happ-enabled\ (t,as)\ s \implies wf-world-model\ s \implies wf-world-model\ (apply-happ\ (t,as)\ s)$
⟨*proof*⟩

corollary *wf-valid-happ-seq:*
assumes *wf-world-model M*
and *wf-happ-seq hs*
and *valid-happ-seq M hs M'*
shows *wf-world-model M'*
⟨*proof*⟩

In a wellformed plan the action parameters match for every plan action

lemma *wf-plan-action-params-match:*
assumes *wf-plan πs*
and $(t,a) \in set\ \pi s$
and $a_{schema} = the\ (resolve-action-schema\ (name\ a))$
shows *action-params-match a_{schema} (arguments a)*
⟨*proof*⟩

lemma *simple-plan-action-schema-type1:*
assumes *wf-plan-action (Simple-Plan-Action $n\ args$)*
shows $\exists\ params\ pre\ eff. resolve-action-schema\ n = Some\ (Simple-Action-Schema\ n\ params\ pre\ eff)$
⟨*proof*⟩

lemma *durative-plan-action-schema-type1:*
assumes *wf-plan-action (Durative-Plan-Action $n\ args\ t'$)*

shows $\exists \text{ params } d \text{ cond eff. resolve-action-schema } n$
 $= \text{Some } (\text{Durative-Action-Schema } n \text{ params } d \text{ cond eff})$
 $\langle \text{proof} \rangle$

Instantiating a wellformed plan action always produces a wellformed ground action

lemma *wf-inst-of-plan-action*:
assumes *wf-plan* πs
and $(t_\pi, \pi) \in \text{set } \pi s$
and *inst-of-plan-action* $\pi s (t_\pi, \pi) (t_i, a)$
shows *wf-ground-action* a
 $\langle \text{proof} \rangle$

Every ground action in an induced happening sequence for a wellformed plan is wellformed.

lemma *happ-seq-wf-ground-action*:
assumes *wf-plan* πs
and *ind-happ-seq* $\pi s hs$
and $(t_i, A_i) \in \text{set } hs$
and $a \in \text{set } A_i$
shows *wf-ground-action* a
 $\langle \text{proof} \rangle$

An induced happening sequence for a wellformed plan is always a wellformed happening sequence.

lemma *inst-wf-plan-wf-happ-seq*:
assumes *wf-plan* πs
and *ind-happ-seq* $\pi s hs$
shows *wf-happ-seq* hs
 $\langle \text{proof} \rangle$

Execution of a plan preserves well-formedness

corollary *wf-plan-happ-path*:
assumes *wf-world-model* M **and** *wf-plan* πs **and** *plan-happ-path* $M \pi s M'$
shows *wf-world-model* M'
 $\langle \text{proof} \rangle$

end — Context of *wf-ast-problem*

Auxiliary function that are used to divide happening sequences.

definition *leq* $\equiv \lambda t_i. \lambda(t_a, as). t_a \leq t_i$

definition *le* $\equiv \lambda t_i. \lambda(t_a, as). t_a < t_i$

lemma *strict-sorted-app-iff*: *strict-sorted* $a \wedge \text{strict-sorted } b \wedge (\forall x \in \text{set } a. \forall y \in \text{set } b. x < y)$
 $\longleftrightarrow \text{strict-sorted } (a @ b)$
 $\langle \text{proof} \rangle$

lemma *empty-filter-conv*: $\llbracket \cdot \rrbracket = \text{filter } P \text{ } xs \longleftrightarrow (\forall x \in \text{set } xs. \neg P \ x)$
 $\langle \text{proof} \rangle$

1.7.3 Completeness of Semantics

context *wf-ast-problem*
begin

fun *consume2* **where**
consume2 $\llbracket \cdot \rrbracket = 0$
 $\mid \text{consume2 } (x \# xs) (y \# ys) = 0 + \text{consume2 } xs \ ys$

lemma *empty-happ*:
 $(\bigwedge a. a \in \text{set } A \implies \text{effect } a = \text{Effect } \llbracket \cdot \rrbracket) \implies \text{apply-happ } (t, A) \ M = M$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-1*:
 $\llbracket \bigwedge t \ A \ a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } \llbracket \cdot \rrbracket;$
 $\text{valid-happ-seq } M \ hs2 \ M';$
 $(\bigwedge t \ A \ a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies M \models_{\text{c}} \text{precondition } a);$
 $(\bigwedge t \ A \ a. (t, A) \in \text{set } hs1 \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket \implies$
 $\text{valid-happ-seq } M \ (hs1 \ @ \ hs2) \ M'$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-2*:
 $\llbracket \text{valid-happ-seq } M \ (hs1 \ @ \ hs2) \ M'; \bigwedge t \ A \ a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies$
 $\text{effect } a = \text{Effect } \llbracket \cdot \rrbracket \rrbracket \implies$
 $\text{valid-happ-seq } M \ hs2 \ M'$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-2'*:
 $\llbracket \text{valid-happ-seq } M \ (hs1 \ @ \ hs2) \ M'; \bigwedge t \ A \ a. \llbracket (t, A) \in \text{set } hs2; a \in \text{set } A \rrbracket \implies$
 $\text{effect } a = \text{Effect } \llbracket \cdot \rrbracket \rrbracket \implies$
 $\text{valid-happ-seq } M \ hs1 \ M'$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-3*:
 $\llbracket (t, A) \in \text{set } hs1; a \in \text{set } A; \text{valid-happ-seq } M \ (hs1 \ @ \ hs2) \ M';$
 $\bigwedge t \ A \ a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } \llbracket \cdot \rrbracket \rrbracket$
 $\implies M \models_{\text{c}} \text{precondition } a$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-4*:
 $\llbracket \text{valid-happ-seq } M \ (hs1 \ @ \ hs2) \ M'; \bigwedge t \ A \ a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies \text{effect}$
 $a = \text{Effect } \llbracket \cdot \rrbracket \rrbracket \implies$
 $(\bigwedge t \ A \ a. (t, A) \in \text{set } hs1 \implies \text{pairwise acts-non-intrf } (\text{set } A))$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-empty-eff*:

$\llbracket \bigwedge t A a. \llbracket (t, A) \in \text{set } hs; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] \rrbracket$;
 $(\bigwedge t A a. \llbracket (t, A) \in \text{set } hs; a \in \text{set } A \rrbracket \implies M \text{ }^c\text{ } \models \text{precondition } a)$;
 $(\bigwedge t A a. (t, A) \in \text{set } hs \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket \implies$
 $\text{valid-happ-seq } M \text{ } hs \text{ } M$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-5*:

$\llbracket \bigwedge t A a. \llbracket (t, A) \in \text{set } hs2; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] \rrbracket$;
 $\text{valid-happ-seq } M \text{ } hs1 \text{ } M'$;
 $(\bigwedge t A a. \llbracket (t, A) \in \text{set } hs2; a \in \text{set } A \rrbracket \implies M' \text{ }^c\text{ } \models \text{precondition } a)$;
 $(\bigwedge t A a. (t, A) \in \text{set } hs2 \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket \implies$
 $\text{valid-happ-seq } M \text{ } (hs1 @ hs2) \text{ } M'$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-cons*: $\llbracket \text{strict-sorted } (\text{map } f \text{ } (x \# xs)); x' \in \text{set } xs \rrbracket \implies f \text{ } x \neq f \text{ } x'$

$\langle \text{proof} \rangle$

lemma *strict-sorted-order*:

$\llbracket (\bigwedge x. x \in s \implies \exists y \in \text{set } (x1 \# xs1 @ x2 \# xs2). f \text{ } y = x); \text{strict-sorted } (\text{map } f \text{ } (x1 \# xs1 @ x2 \# xs2)) \rrbracket$;
 $(\bigwedge y. y \in \text{set } xs1 \implies f \text{ } y \notin s); x \in s; x < f \text{ } x2 \rrbracket \implies x = f \text{ } x1$
 $\langle \text{proof} \rangle$

lemma *in-suff*:

$\llbracket \bigwedge x. x \in \text{set } xs1 \implies x \neq x2; x2 \in \text{set } (xs1 @ xs2) \rrbracket \implies x2 \in \text{set } xs2$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-order-in-suff*:

$\llbracket \text{strict-sorted } (xs1 @ xs2); x1 \in \text{set } xs1; x2 \in \text{set } xs2 \rrbracket \implies x1 < x2$
 $\langle \text{proof} \rangle$

lemma *empty-eff-non-intrf*:

$\llbracket \bigwedge a. a \in A \implies \text{effect } a = \text{Effect } [] \rrbracket \implies \text{pairwise acts-non-intrf } A$
 $\langle \text{proof} \rangle$

lemma *filter-eq-consD*: $x \# l' = \text{filter } Q \text{ } l \implies x \in \text{set } l \wedge Q \text{ } x$

$\langle \text{proof} \rangle$

lemma *valid-hap-seq-construct*:

assumes $\bigwedge t. t \in \text{htps} \implies \exists A. (t, A) \in \text{set } hs$
and $\bigwedge t. t \in \text{htps} \implies \exists A. (t, A) \in \text{set } hs'$
and $\text{strict-sorted } (\text{map } \text{fst } hs)$
and $\text{strict-sorted } (\text{map } \text{fst } hs')$
and $\bigwedge t A A'. \llbracket t \in \text{htps}; (t, A) \in \text{set } hs; (t, A') \in \text{set } hs' \rrbracket \implies \text{set } A = \text{set } A'$
and $\bigwedge t A A' a. \llbracket t \notin \text{htps}; (t, A) \in \text{set } hs; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } []$
and $\bigwedge t A A' a. \llbracket t \notin \text{htps}; (t, A) \in \text{set } hs'; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } []$
and $\bigwedge t A t_1 t_2 a.$

$\llbracket t \notin \text{https}; t_1 \in \text{https}; t_2 \in \text{https}; t \in \{t_1 < \dots < t_2\}; (t, A) \in \text{set } \text{hs}; a \in \text{set } A \rrbracket \implies$
 $\quad \exists t' \in \{t_1 < \dots < t_2\}. \exists A'. (t', A') \in \text{set } \text{hs}' \wedge a \in \text{set } A'$
and $\bigwedge t A t_1 t_2 a.$
 $\llbracket t \notin \text{https}; t_1 \in \text{https}; t_2 \in \text{https}; t \in \{t_1 < \dots < t_2\}; (t, A) \in \text{set } \text{hs}'; a \in \text{set } A \rrbracket \implies$
 $\quad \exists t' \in \{t_1 < \dots < t_2\}. \exists A'. (t', A') \in \text{set } \text{hs} \wedge a \in \text{set } A'$
and *valid-happ-seq* $M \text{ hs } M'$
and $\text{hs} \neq [] \implies (\text{fst } (\text{hd } \text{hs})) \in \text{https}$
and $\text{hs}' \neq [] \implies (\text{fst } (\text{hd } \text{hs}')) \in \text{https}$
and $\text{hs} \neq [] \implies (\text{fst } (\text{last } \text{hs})) \in \text{https}$
and $\text{hs}' \neq [] \implies (\text{fst } (\text{last } \text{hs}')) \in \text{https}$
shows *valid-happ-seq* $M \text{ hs}' M'$
 $\langle \text{proof} \rangle$

lemma *not-htp-empty-effect*:

assumes *wf-plan* πs
and $\neg \text{is-htp } \pi s t_i$
and *ind-happ-seq* $\pi s \text{ hs}$
and $(t_i, A_i) \in \text{set } \text{hs}$
and $a \in \text{set } A_i$
shows $(\text{effect } a) = \text{Effect } [] []$
 $\langle \text{proof} \rangle$

lemma *unique-happ*:

assumes *ind-happ-seq* $\pi s \text{ hs}$
shows $\llbracket (t, A) \in \text{set } \text{hs}; (t, A') \in \text{set } \text{hs} \rrbracket \implies A = A'$
 $\langle \text{proof} \rangle$

lemma *inst-of-plan-actionE*:

$\bigwedge t_\pi \pi. \llbracket \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_a, a);$
 $\llbracket \text{res-inst } \pi = \text{Some } a; t_\pi = t_a \rrbracket \implies Q;$
 $\llbracket \text{res-inst-snap-action } \pi \text{ At-Start} = \text{Some } a; t_\pi = t_a \rrbracket \implies Q;$
 $\llbracket \text{res-inst-snap-action } \pi \text{ At-End} = \text{Some } a; t_\pi + (\text{duration } \pi) = t_a \rrbracket \implies Q;$
 $\bigwedge t_i t_j. \llbracket \text{res-inst-snap-action } \pi \text{ Over-All} = \text{Some } a; \text{consec-https } \pi s t_i t_j;$
 $t_\pi \leq t_i; t_j \leq t_\pi + (\text{duration } \pi); t_i < t_a; t_a < t_j \rrbracket \implies Q \rrbracket$
 $\implies Q$
 $\langle \text{proof} \rangle$

lemma *ind-happ-seq-happ-at-https*:

assumes *is-htp* $\pi s t$
and *ind-happ-seq* $\pi s \text{ hs}$
and *ind-happ-seq* $\pi s \text{ hs}'$
and $(t, A) \in \text{set } \text{hs}$
and $(t, A') \in \text{set } \text{hs}'$
and $a \in \text{set } A$
shows $a \in \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *ind-happ-seq-happ-at-htps'*:

assumes *is-htp* $\pi s\ t$
and *ind-happ-seq* $\pi s\ hs$
and *ind-happ-seq* $\pi s\ hs'$
and $(t, A) \in \text{set } hs$
and $(t, A') \in \text{set } hs'$
shows $\text{set } A = \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *htp-in-ind-happ-seq*:

$\llbracket \text{is-htp } \pi s\ t; \text{ind-happ-seq } \pi s\ hs \rrbracket \implies \exists A. (t, A) \in \text{set } hs$
 $\langle \text{proof} \rangle$

lemma *sorted-takeWhile*: $\text{strict-sorted } xs \implies x \in \text{set } (\text{takeWhile } ((\lambda x. x < c))\ xs)$
 $= (x \in \text{set } xs \wedge (x < c))$
 $\langle \text{proof} \rangle$

lemma *sorted-dropWhile*: $\text{strict-sorted } xs \implies x \in \text{set } (\text{dropWhile } ((\lambda x. x \leq c))\ xs)$
 $= (x \in \text{set } xs \wedge (c < x))$
 $\langle \text{proof} \rangle$

lemma *map-fst-dropWhile*: $\text{map } \text{fst } (\text{dropWhile } (\text{leq } t)\ hs) = (\text{dropWhile } ((\lambda x. x \leq t))\ (\text{map } \text{fst } hs))$
 $\langle \text{proof} \rangle$

lemma *map-fst-takeWhile*: $\text{map } \text{fst } (\text{takeWhile } (\text{le } t)\ hs) = (\text{takeWhile } ((\lambda x. x < t))\ (\text{map } \text{fst } hs))$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-fst-unique-snd*:

$\llbracket \text{strict-sorted } (\text{map } \text{fst } hs); (t, A) \in \text{set } hs; (t, A') \in \text{set } hs \rrbracket \implies A = A'$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-dropWhile*:

$\text{strict-sorted } xs \implies \text{strict-sorted } (\text{dropWhile } Q\ xs)$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-takeWhile*:

$\text{strict-sorted } xs \implies \text{strict-sorted } (\text{takeWhile } Q\ xs)$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-drop-leq-take-le'*:

assumes *strict-sorted* $(\text{map } \text{fst } hs)$
shows $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i)\ (\text{takeWhile } (\text{le } t_j)\ hs)) \longleftrightarrow$
 $(t_i < t_a \wedge t_a < t_j \wedge (t_a, A) \in \text{set } hs) \text{ (is } ?l \longleftrightarrow ?r)$
 $\langle \text{proof} \rangle$

lemma *ind-happ-seqs-same-acts-leq_i-le_j'*:

assumes *is-htp* $\pi s t_i$
and *is-htp* $\pi s t_j$
and *ind-happ-seq* $\pi s hs_1$
and *ind-happ-seq* $\pi s hs_2$
and $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_1))$
and $a \in \text{set } A$
shows $\exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_2)) \wedge a$
 $\in \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *happs-same-between-htps*:

assumes *ind-happ-seq* $\pi s hs$
and *ind-happ-seq* $\pi s hs'$
and *is-htp* $\pi s t_1$ *is-htp* $\pi s t_2$
and $(t, A) \in \text{set } hs$
and $\neg \text{is-htp } \pi s t$
and $t_1 < t$
and $t < t_2$
and $a \in \text{set } A$
shows $\exists t' \in \{t_1 < .. < t_2\}. \exists A'. (t', A') \in \text{set } hs' \wedge a \in \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *consec-htps-props*:

assumes *consec-htps* $\pi s t_i t_j$
shows $t_i < t_j$
is-htp $\pi s t_i$
is-htp $\pi s t_j$
is-htp $\pi s t \implies t \leq t_i \vee t_j \leq t$
 $\langle \text{proof} \rangle$

lemma *ind-happ-seq-hd-htp*:

assumes *ind-happ-seq* $\pi s hs$ $hs \neq []$
shows *is-htp* $\pi s (\text{fst } (\text{hd } hs))$
 $\langle \text{proof} \rangle$

lemma *sorted-last*: $\llbracket \text{strict-sorted } xs; x \in \text{set } xs \rrbracket \implies x \leq \text{last } xs$
 $\langle \text{proof} \rangle$

lemma *ind-happ-seq-last-htp*:

assumes *ind-happ-seq* $\pi s hs$ $hs \neq []$
shows *is-htp* $\pi s (\text{fst } (\text{last } hs))$
 $\langle \text{proof} \rangle$

definition *htps'* $\pi s \equiv \{t . \text{is-htp } \pi s t\}$

lemma *htps-props*: $t \in \text{htps'} \pi s \implies \text{is-htp } \pi s t$
 $\langle \text{proof} \rangle$

lemma *ind-happ-seq-unique-final-state'*:

```

assumes ind-happ-seq  $\pi s$  hs and ind-happ-seq  $\pi s$  hs'
shows  $\llbracket wf-plan \ \pi s; \ valid-happ-seq \ M_0 \ hs \ M_n \rrbracket \implies \ valid-happ-seq \ M_0 \ hs' \ M_n$ 
 $\langle proof \rangle$ 

end

end — Theory
theory Error-Monad-Add
imports
  Certification-Monads.Check-Monad
  Show.Show-Instances
begin

  abbreviation assert-opt  $\Phi \equiv$  if  $\Phi$  then Some  $()$  else None

  definition lift-opt  $m \ e \equiv$  case  $m$  of Some  $x \Rightarrow$  Error-Monad.return  $x$  | None  $\Rightarrow$ 
Error-Monad.error  $e$ 

  lemma lift-opt-simps[simp]:
    lift-opt None  $e =$  error  $e$ 
    lift-opt (Some  $v$ )  $e =$  return  $v$ 
     $\langle proof \rangle$ 

  lemma reflcl-image-iff[simp]:  $R = "S = S \cup R"$   $\langle proof \rangle$ 

  named-theorems return-iff

  lemma bind-return-iff[return-iff]: Error-Monad.bind  $m \ f =$  Inr  $y \longleftrightarrow (\exists x. \ m$ 
 $=$  Inr  $x \wedge f \ x =$  Inr  $y)$ 
     $\langle proof \rangle$ 

  lemma lift-opt-return-iff[return-iff]: lift-opt  $m \ e =$  Inr  $x \longleftrightarrow m =$  Some  $x$ 
     $\langle proof \rangle$ 

  lemma check-return-iff[return-iff]: check  $\Phi \ e =$  Inr  $uu \longleftrightarrow \Phi$ 
     $\langle proof \rangle$ 

  lemma check-simps[simp]:
    check True  $e =$  succeed
    check False  $e =$  error  $e$ 
     $\langle proof \rangle$ 

  lemma Let-return-iff[return-iff]: (let  $x=v$  in  $f \ x$ )  $=$  Inr  $w \longleftrightarrow f \ v =$  Inr  $w$   $\langle proof \rangle$ 

  abbreviation ERR  $::$  shows  $\Rightarrow$  (unit  $\Rightarrow$  shows) where ERR  $s \equiv \lambda -. \ s$ 
  abbreviation ERRS  $::$  string  $\Rightarrow$  (unit  $\Rightarrow$  shows) where ERRS  $s \equiv$  ERR (shows

```

s)

lemma *error-monad-bind-split*: $P \text{ (bind } m \text{ f)} \longleftrightarrow (\forall v. m = \text{Inl } v \longrightarrow P \text{ (Inl } v))$
 $\wedge (\forall v. m = \text{Inr } v \longrightarrow P \text{ (f } v))$
 $\langle \text{proof} \rangle$

lemma *error-monad-bind-split-asm*: $P \text{ (bind } m \text{ f)} \longleftrightarrow \neg (\exists x. m = \text{Inl } x \wedge \neg P \text{ (f } x))$
 $(\text{Inl } x) \vee (\exists x. m = \text{Inr } x \wedge \neg P \text{ (f } x))$
 $\langle \text{proof} \rangle$

lemmas *error-monad-bind-splits = error-monad-bind-split error-monad-bind-split-asm*

end

2 Executable Temporal PDDL Checker

theory *TEMPORAL-PDDL-Checker*

imports

TEMPORAL-PDDL-Semantics

Error-Monad-Add

HOL.String

HOL-Library.Code-Target-Nat

HOL-Library.While-Combinator

Containers.Containers

HOL-Library.Tree

begin

2.1 Generic DFS Reachability Checker

Used for subtype checks

definition *E-of-succ succ* $\equiv \{ (u, v). v \in \text{set } (\text{succ } u) \}$

lemma *succ-as-E*: $\text{set } (\text{succ } x) = E\text{-of-succ succ } \{x\}$
 $\langle \text{proof} \rangle$

context

fixes *succ* :: 'a $\Rightarrow$ 'a list

begin

private abbreviation *(input) E* $\equiv E\text{-of-succ succ}$

definition *dfs-reachable D w* $\equiv$

$\text{let } (V, w, brk) = \text{while } (\lambda (V, w, brk). \neg brk \wedge w \neq []) (\lambda (V, w, -). \text{case } w \text{ of } v \# w \Rightarrow$


```

    if D v then (V, v # w, True)
    else if v ∈ V then (V, w, False)
    else
      let V = insert v V in
      let w = succ v @ w in
      (V, w, False)
  ) ({} , w, False)
in brk

```

context

```

  fixes w0 :: 'a list
  assumes finite-dfs-reachable[simp, intro!]: finite (E* “ set w0)
begin

```

private abbreviation (input) $W_0 \equiv \text{set } w_0$

definition *dfs-reachable-invar* $D \ V \ W \ brk \longleftrightarrow$

```

  W0 ⊆ W ∪ V
  ∧ W ∪ V ⊆ E* “ W0
  ∧ E “ V ⊆ W ∪ V
  ∧ Collect D ∩ V = {}
  ∧ (brk ⟶ Collect D ∩ E* “ W0 ≠ {})

```

lemma *card-decreases*:

```

  ⟦finite V; y ∉ V; dfs-reachable-invar D V (Set.insert y W) brk ⟧
  ⟹ card (E* “ W0 - Set.insert y V) < card (E* “ W0 - V)
⟨proof⟩

```

lemma *all-neq-Cons-is-Nil*[simp]:

```

  (∀ y ys. x2 ≠ y # ys) ⟷ x2 = [] ⟨proof⟩

```

lemma *dfs-reachable-correct*: $\text{dfs-reachable } D \ w_0 \longleftrightarrow \text{Collect } D \cap E^* \text{ “ set } w_0 \neq \{\}$

$\{\}$
 ⟨proof⟩

end

definition *tab-succ* $l \equiv \text{Mapping.lookup-default } [] \ (\text{fold } (\lambda(u,v). \text{Mapping.map-default } u \ [] \ (\text{Cons } v)) \ l \ \text{Mapping.empty})$

lemma *Some-eq-map-option* [iff]: $(\text{Some } y = \text{map-option } f \ x_0) = (\exists z. x_0 = \text{Some } z \wedge f \ z = y)$

⟨proof⟩

lemma *tab-succ-correct*: $E\text{-of-succ } (\text{tab-succ } l) = \text{set } l$

⟨proof⟩

end

lemma *finite-imp-finite-dfs-reachable*:
 $\llbracket \text{finite } E; \text{finite } S \rrbracket \implies \text{finite } (E^* \text{ `` } S)$
 $\langle \text{proof} \rangle$

lemma *dfs-reachable-tab-succ-correct*: $\text{dfs-reachable } (\text{tab-succ } l) \ D \ vs_0 \longleftrightarrow \text{Collect } D \cap (\text{set } l)^* \text{ `` } \text{set } vs_0 \neq \{\}$
 $\langle \text{proof} \rangle$

2.2 Implementation Refinements

2.2.1 Of-Type

definition *of-type-impl* $G \ oT \ T \equiv (\forall pt \in \text{set } (\text{primitives } oT). \text{dfs-reachable } G \ ((=) \ pt) \ (\text{primitives } T))$

fun *ty-term'* **where**
 $\text{ty-term'} \ \text{varT} \ \text{objT} \ (\text{term.VAR } v) = \text{varT } v$
 $| \text{ty-term'} \ \text{varT} \ \text{objT} \ (\text{term.CONST } c) = \text{Mapping.lookup } \text{objT} \ c$

lemma *ty-term'-correct-aux*: $\text{ty-term'} \ \text{varT} \ \text{objT} \ t = \text{ty-term } \text{varT} \ (\text{Mapping.lookup } \text{objT}) \ t$
 $\langle \text{proof} \rangle$

lemma *ty-term'-correct[simp]*: $\text{ty-term'} \ \text{varT} \ \text{objT} = \text{ty-term } \text{varT} \ (\text{Mapping.lookup } \text{objT})$
 $\langle \text{proof} \rangle$

context *ast-domain* **begin**

definition *of-type1* $pt \ T \longleftrightarrow pt \in \text{subtype-rel}^* \text{ `` } \text{set } (\text{primitives } T)$

lemma *of-type-refine1*: $\text{of-type } oT \ T \longleftrightarrow (\forall pt \in \text{set } (\text{primitives } oT). \text{of-type1 } pt \ T)$
 $\langle \text{proof} \rangle$

definition *STG* $\equiv (\text{tab-succ } (\text{map } \text{subtype-edge } (\text{types } D)))$

lemma *subtype-rel-impl*: $\text{subtype-rel} = E\text{-of-succ } (\text{tab-succ } (\text{map } \text{subtype-edge } (\text{types } D)))$
 $\langle \text{proof} \rangle$

lemma *of-type1-impl*: $\text{of-type1 } pt \ T \longleftrightarrow \text{dfs-reachable } (\text{tab-succ } (\text{map } \text{subtype-edge } (\text{types } D))) \ ((=) \ pt) \ (\text{primitives } T)$
 $\langle \text{proof} \rangle$

lemma *of-type-impl-correct*: $\text{of-type-impl } STG \ oT \ T \longleftrightarrow \text{of-type } oT \ T$

$\langle \text{proof} \rangle$

definition $\text{mp-constT} :: (\text{object}, \text{type}) \text{ mapping where}$
 $\text{mp-constT} = \text{Mapping.of-alist } (\text{consts } D)$

lemma $\text{mp-objT-correct[simp]}: \text{Mapping.lookup mp-constT} = \text{constT}$
 $\langle \text{proof} \rangle$

Lifting the subtype-graph through wf-checker

context

fixes $\text{ty-ent} :: 'ent \rightarrow \text{type} \text{ --- Entity's type, None if invalid}$

begin

definition $\text{is-of-type}' \text{ stg } v \text{ } T \longleftrightarrow ($
 $\text{case ty-ent } v \text{ of}$
 $\text{Some } vT \Rightarrow \text{of-type-impl stg } vT \text{ } T$
 $| \text{None} \Rightarrow \text{False})$

lemma $\text{is-of-type}'\text{-correct}: \text{is-of-type}' \text{ STG } v \text{ } T = \text{is-of-type ty-ent } v \text{ } T$
 $\langle \text{proof} \rangle$

fun $\text{wf-pred-atom}' \text{ where } \text{wf-pred-atom}' \text{ stg } (p, vs) \longleftrightarrow$
 $(\text{case sig } p \text{ of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } Ts \Rightarrow \text{list-all2 } (\text{is-of-type}' \text{ stg}) \text{ } vs \text{ } Ts)$

lemma $\text{wf-pred-atom}'\text{-correct}: \text{wf-pred-atom}' \text{ STG } pvs = \text{wf-pred-atom ty-ent}$
 pvs
 $\langle \text{proof} \rangle$

fun $\text{wf-func-args}' \text{ where } \text{wf-func-args}' \text{ stg } (f, args) \longleftrightarrow$
 $(\text{case func-sig } f \text{ of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } Ts \Rightarrow \text{list-all2 } (\text{is-of-type}' \text{ stg}) \text{ } args \text{ } Ts)$

lemma $\text{wf-func-args}'\text{-correct}: \text{wf-func-args}' \text{ STG } fargs = \text{wf-func-args ty-ent}$
 $fargs$
 $\langle \text{proof} \rangle$

fun $\text{wf-atom}' :: - \Rightarrow 'ent \text{ atom} \Rightarrow \text{bool where}$
 $\text{wf-atom}' \text{ stg } (\text{atom.predAtm } p \text{ } vs) \longleftrightarrow \text{wf-pred-atom}' \text{ stg } (p, vs)$
 $| \text{wf-atom}' \text{ stg } (\text{atom.eqAtm } a \text{ } b) = (\text{ty-ent } a \neq \text{None} \wedge \text{ty-ent } b \neq \text{None})$

lemma $\text{wf-atom}'\text{-correct}: \text{wf-atom}' \text{ STG } a = \text{wf-atom ty-ent } a$
 $\langle \text{proof} \rangle$

fun $\text{wf-fmla}' :: - \Rightarrow ('ent \text{ atom}) \text{ formula} \Rightarrow \text{bool where}$
 $\text{wf-fmla}' \text{ stg } (\text{Atom } a) \longleftrightarrow \text{wf-atom}' \text{ stg } a$
 $| \text{wf-fmla}' \text{ stg } \perp \longleftrightarrow \text{True}$

```

| wf-fmla' stg ( $\varphi 1 \wedge \varphi 2$ )  $\longleftrightarrow$  (wf-fmla' stg  $\varphi 1 \wedge$  wf-fmla' stg  $\varphi 2$ )
| wf-fmla' stg ( $\varphi 1 \vee \varphi 2$ )  $\longleftrightarrow$  (wf-fmla' stg  $\varphi 1 \wedge$  wf-fmla' stg  $\varphi 2$ )
| wf-fmla' stg ( $\varphi 1 \rightarrow \varphi 2$ )  $\longleftrightarrow$  (wf-fmla' stg  $\varphi 1 \wedge$  wf-fmla' stg  $\varphi 2$ )
| wf-fmla' stg ( $\neg \varphi$ )  $\longleftrightarrow$  wf-fmla' stg  $\varphi$ 

```

lemma wf-fmla'-correct: wf-fmla' STG $\varphi \longleftrightarrow$ wf-fmla ty-ent φ
 $\langle \text{proof} \rangle$

fun wf-fmla-atom1' **where**
 wf-fmla-atom1' stg (Atom (predAtm p vs)) $\longleftrightarrow$ wf-pred-atom' stg (p,vs)
 | wf-fmla-atom1' stg - $\longleftrightarrow$ False

lemma wf-fmla-atom1'-correct: wf-fmla-atom1' STG $\varphi =$ wf-fmla-atom ty-ent φ
 $\langle \text{proof} \rangle$

fun wf-effect' **where**
 wf-effect' stg (Effect a d) $\longleftrightarrow$
 ($\forall ae \in \text{set } a. \text{wf-fmla-atom1' stg } ae$)
 $\wedge (\forall de \in \text{set } d. \text{wf-fmla-atom1' stg } de)$

lemma wf-effect'-correct: wf-effect' STG $e =$ wf-effect ty-ent e
 $\langle \text{proof} \rangle$

fun wf-duration-const' :: - $\Rightarrow$ 'ent duration-constraint $\Rightarrow$ bool **where**
 wf-duration-const' stg No-Const $\longleftrightarrow$ True
 | wf-duration-const' stg (Time-Const op d) $\longleftrightarrow d \geq 0$
 | wf-duration-const' stg (Func-Const op f vs) $\longleftrightarrow$
 (case func-sig f of
 None $\Rightarrow$ False
 | Some Ts $\Rightarrow$ list-all2 (is-of-type' stg) vs Ts)

lemma wf-duration-const'-correct: wf-duration-const' STG $d =$ wf-duration-const ty-ent d
 $\langle \text{proof} \rangle$

end — Context fixing ty-ent

fun wf-action-schema' :: - $\Rightarrow$ - $\Rightarrow$ ast-action-schema $\Rightarrow$ bool **where**
 wf-action-schema' stg conT (Simple-Action-Schema n params pre eff) $\longleftrightarrow$ (
 let
 tyv = ty-term' (map-of params) conT
 in
 distinct (map fst params)
 $\wedge$ wf-fmla' tyv stg pre
 $\wedge$ wf-effect' tyv stg eff)
 | wf-action-schema' stg conT (Durative-Action-Schema n params d cond eff) $\longleftrightarrow$
 (
 let

$tyv = ty-term' (map-of params) conT$
in
 $distinct (map fst params)$
 $\wedge (\forall (t,c) \in set\ cond. wf-fmla' tyv stg c)$
 $\wedge (\forall (t,e) \in set\ eff. wf-effect' tyv stg e \wedge t \neq Over-All)$
 $\wedge wf-duration-const' tyv stg d)$

lemma *wf-action-schema'-correct*: $wf-action-schema' STG mp-constT s = wf-action-schema$
 s
 $\langle proof \rangle$

definition *wf-domain'* :: $- \Rightarrow - \Rightarrow bool$ **where**
 $wf-domain' stg conT \equiv$
 $wf-types$
 $\wedge distinct (map (predicate-decl.pred) (predicates D))$
 $\wedge (\forall p \in set (predicates D). wf-predicate-decl p)$
 $\wedge distinct (map (function-decl.func) (functions D))$
 $\wedge (\forall f \in set (functions D). wf-function-decl f)$
 $\wedge distinct (map fst (consts D))$
 $\wedge (\forall (n,T) \in set (consts D). wf-type T)$
 $\wedge distinct (map ast-action-schema.name (actions D))$
 $\wedge (\forall a \in set (actions D). wf-action-schema' stg conT a)$

lemma *wf-domain'-correct*: $wf-domain' STG mp-constT = wf-domain$
 $\langle proof \rangle$

end — Context of *ast-domain*

2.2.2 Ground Action Interference & Application of Effects

context *ast-problem* **begin**

Implementation of executable refinements for *acts-non-intrf* & *apply-happ*

a list of simultaneous & non-interfering grounded actions are all applied at once

fun *apply-happ-exec* :: $happening \Rightarrow world-model \Rightarrow world-model$ **where**
 $apply-happ-exec (t_i, A_i) s =$
 $(let$
 $adds = (concat\ o\ (map\ (adds\ o\ effect)))\ A_i;$
 $dels = (concat\ o\ (map\ (dels\ o\ effect)))\ A_i\ in$
 $fold\ Set.insert\ adds\ (fold\ Set.remove\ dels\ s))$

lemma *fold-set-remove*: $fold\ Set.remove\ xs\ s = s - set\ xs$
 $\langle proof \rangle$

lemma *fold-set-insert*: $fold\ Set.insert\ xs\ s = s \cup set\ xs$

$\langle \text{proof} \rangle$

Justification of refinement for *apply-happ*

lemma *apply-happ-exec-refine*: *apply-happ-exec h s = apply-happ h s*
 $\langle \text{proof} \rangle$

end — Context of *ast-problem*

2.2.3 Well-Formedness

context *ast-problem* **begin**

We start by defining a mapping from objects to types. The container framework will generate efficient, red-black tree based code for that later.

type-synonym *objT* = (*object*, *type*) *mapping*

definition *mp-objT* :: (*object*, *type*) *mapping* **where**
mp-objT = *Mapping.of-alist* (*consts D @ objects P*)

lemma *mp-objT-correct[simp]*: *Mapping.lookup mp-objT = objT*
 $\langle \text{proof} \rangle$

We refine the typecheck to use the mapping

definition *is-obj-of-type-impl stg mp n T* = (
case Mapping.lookup mp n of None $\Rightarrow$ False | Some oT $\Rightarrow$ of-type-impl stg oT
T
)

lemma *is-obj-of-type-impl-correct[simp]*:
is-obj-of-type-impl STG mp-objT = is-obj-of-type
 $\langle \text{proof} \rangle$

fun *wf-func-assign'* :: *objT* $\Rightarrow$ - $\Rightarrow$ *object atom formula* $\Rightarrow$ *bool* **where**
wf-func-assign' ot stg (*Atom* (*eqAtm* (*FuncEnt f args*) (*TimeEnt d*)))
 $\longleftrightarrow$ *wf-func-args'* (*Mapping.lookup ot*) *stg* (*f, args*)
| *wf-func-assign' ot stg* - $\longleftrightarrow$ *False*

lemma *wf-func-assign'-correct[simp]*: *wf-func-assign' mp-objT STG f = wf-func-assign f*
 $\langle \text{proof} \rangle$

We refine the well-formedness checks to use the mapping

definition *wf-fact'* :: *objT* $\Rightarrow$ - $\Rightarrow$ *fact* $\Rightarrow$ *bool* **where**
wf-fact' ot stg $\equiv$ *wf-pred-atom'* (*Mapping.lookup ot*) *stg*

lemma *wf-fact'-correct[simp]*: *wf-fact' mp-objT STG = wf-fact*
 $\langle \text{proof} \rangle$

definition *wf-fmla-atom2'* mp stg f
 $= (\text{case } f \text{ of formula.Atom } (\text{predAtm } p \text{ vs}) \Rightarrow (\text{wf-fact}' \text{ mp stg } (p, \text{vs})) \mid - \Rightarrow \text{False})$

lemma *wf-fmla-atom2'-correct[simp]*:
 $\text{wf-fmla-atom2}' \text{ mp-objT STG } \varphi = \text{wf-fmla-atom objT } \varphi$
 <proof>

definition *wf-problem'* stg conT mp $\equiv$
 $\text{wf-domain}' \text{ stg conT}$
 $\wedge \text{distinct } (\text{map fst } (\text{objects } P) \text{ @ } \text{map fst } (\text{consts } D))$
 $\wedge (\forall (n, T) \in \text{set } (\text{objects } P). \text{wf-type } T)$
 $\wedge \text{distinct } (\text{init } P)$
 $\wedge (\forall f \in \text{set } (\text{init } P). \text{wf-fmla-atom2}' \text{ mp stg } f \vee \text{wf-func-assign}' \text{ mp stg } f)$
 $\wedge \text{wf-fmla}' (\text{Mapping.lookup } \text{mp}) \text{ stg } (\text{goal } P)$

lemma *wf-problem'-correct*:
 $\text{wf-problem}' \text{ STG mp-constT mp-objT} = \text{wf-problem}$
 <proof>

Instantiating actions will yield well-founded effects. Corollary of $\llbracket \text{action-params-match} (\text{Simple-Action-Schema } ?n \text{ ?params ?pre ?eff}) \text{ ?args}; \text{wf-action-schema } (\text{Simple-Action-Schema } ?n \text{ ?params ?pre ?eff}) \rrbracket \implies \text{wf-ground-action } (\text{instantiate-action-schema } (\text{Simple-Action-Schema } ?n \text{ ?params ?pre ?eff}) \text{ ?args})$ and $\llbracket \text{action-params-match} (\text{Durative-Action-Schema } ?n \text{ ?params ?d ?cond ?eff}) \text{ ?args}; \text{wf-action-schema } (\text{Durative-Action-Schema } ?n \text{ ?params ?d ?cond ?eff}) \rrbracket \implies \text{wf-ground-action } (\text{inst-snap-action } (\text{Durative-Action-Schema } ?n \text{ ?params ?d ?cond ?eff}) \text{ ?args ?ta})$.

lemma *wf-effect-inst-weak*:
fixes n params pre eff
defines $a_{\text{schema}} \equiv \text{Simple-Action-Schema } n \text{ params pre eff}$
assumes $a = \text{instantiate-action-schema } a_{\text{schema}} \text{ args}$
and $\text{action-params-match } a_{\text{schema}} \text{ args}$
and $\text{wf-action-schema } a_{\text{schema}}$
shows $\text{wf-effect-inst } (\text{effect } a)$
 <proof>

lemma *wf-effect-durative-inst-weak*:
fixes n params d cond eff
defines $a_{\text{schema}} \equiv \text{Durative-Action-Schema } n \text{ params d cond eff}$
assumes $a = \text{inst-snap-action } a_{\text{schema}} \text{ args ta}$
and $\text{action-params-match } a_{\text{schema}} \text{ args}$
and $\text{wf-action-schema } a_{\text{schema}}$
shows $\text{wf-effect-inst } (\text{effect } a)$
 <proof>

end — Context of *ast-problem*

2.2.4 Happening Execution

context *ast-domain* **begin**

We first lift action schema lookup into the error monad.

definition *resolve-action-schemaE* *n* $\equiv$
 lift-opt
 $(\text{resolve-action-schema } n)$
 $(\text{ERR } (\text{shows "No such action schema " } o \text{ shows } n))$

lemma *resolve-action-schemaE-return-iff*[*return-iff*]:
 $(\text{resolve-action-schemaE } n = \text{Inr } a) \longleftrightarrow (\text{resolve-action-schema } n = \text{Some } a)$
 $\langle \text{proof} \rangle$

end — Context of *ast-domain*

context *ast-problem* **begin**

We define a function to determine whether a formula holds in a world model

definition *holds* *M* *F* $\equiv (\text{valuation } M) \models F$

Justification of this function

lemma *holds-for-wf-fmlas*:
assumes *wm-basic* *s*
shows *holds* *s* *F* $\longleftrightarrow$ *close-world* *s* $\models F$
 $\langle \text{proof} \rangle$

The first refinement summarizes the enabledness check and the application of a happening. Moreover, we implement the precondition evaluation by our *holds* function. This way, we can eliminate redundant resolving and instantiation of the action.

definition *en-exE* :: *happening* $\Rightarrow$ *world-model* $\Rightarrow$ -+ *world-model* **where**
 $\text{en-exE} \equiv \lambda(t_i, A_i) \Rightarrow \lambda s. \text{do } \{$
 $\text{check-allm } (\lambda a. \text{check } (\text{holds } s \text{ (precondition } a)) \text{ (ERRS "Precondition not satisfied")}) A_i;$
 $\text{check-pairwise } (\lambda a b. \text{check } (a \neq b \longrightarrow \text{acts-non-intrf } a b) \text{ (ERRS "Actions in happening interfering")}) A_i;$
 $\text{Error-Monad.return } (\text{apply-happ-exec } (t_i, A_i) s)$
 $\}$

lemma *symmetric-pred-check-pairwise*:
assumes $\forall x_1 x_2. A x_1 x_2 \longleftrightarrow A x_2 x_1$
and *isOK* $(\text{check-pairwise } (\lambda x_1 x_2. \text{check } (x_1 \neq x_2 \longrightarrow A x_1 x_2) e) as)$
shows $\forall x_1 \in \text{set } as. \forall x_2 \in \text{set } as. x_1 \neq x_2 \longrightarrow A x_1 x_2$
 $\langle \text{proof} \rangle$

Justification of implementation.

lemma (**in** *wf-ast-problem*) *en-exE-return-iff*:

assumes *wm-basic s*
and $\forall a \in \text{set } A_i. \text{ wf-ground-action } a$
shows $\text{en-exE } (t_i, A_i) \ s = \text{Inr } s' \iff \text{happ-enabled } (t_i, A_i) \ s \wedge s' = \text{apply-happ}$
 $(t_i, A_i) \ s$
 $\langle \text{proof} \rangle$

Next, we use the efficient implementation *is-obj-of-type-impl* for the type check, and omit the well-formedness check, as effects obtained from instantiating well-formed action schemas are always well-formed (*wf-effect-inst-weak*).

abbreviation *action-params-match2 stg mp a args*
 $\equiv \text{list-all2 } (\text{is-obj-of-type-impl } \text{stg } \text{mp})$
 $\text{args } (\text{map } \text{snd } (\text{ast-action-schema.parameters } a))$

end — Context of *ast-problem*

2.2.5 Construction of Induced Happening Sequence

context *ast-problem begin*

implemented own version of *insort-insert*, because *insort-insert* requires sort type; rat is not of sort type

fun *insort-htp* :: *time* $\Rightarrow$ *time list* $\Rightarrow$ *time list* **where**
 $\text{insort-htp } x \ [] = [x]$
 $| \text{insort-htp } x \ (y \# \text{ys}) =$
 $\quad (\text{if } x < y \text{ then } x \# y \# \text{ys}$
 $\quad \text{else if } x = y \text{ then } y \# \text{ys}$
 $\quad \text{else } y \# (\text{insort-htp } x \ \text{ys}))$

Basic properties text of *insort-htp*: preserves sorted-ness & distinct-ness, inserts element

lemma *insort-htps-insort-insert*: $\text{sorted } xs \implies \text{insort-htp } x \ xs = \text{insort-insert } x$
 xs
 $\langle \text{proof} \rangle$

lemma *sorted-insort-htps*: $\text{sorted } xs \implies \text{sorted } (\text{insort-htp } x \ xs)$
 $\langle \text{proof} \rangle$

lemma *set-insort-htps*: $\text{set } (\text{insort-htp } x \ xs) = \text{insert } x \ (\text{set } xs)$
 $\langle \text{proof} \rangle$

lemma *distinct-insort-htps*: $\text{sorted } xs \implies \text{distinct } xs \implies \text{distinct } (\text{insort-htp } x$
 $xs)$
 $\langle \text{proof} \rangle$

lemma *insort-htps-non-Nil*: $\text{insort-htp } x \ xs \neq []$
 $\langle \text{proof} \rangle$

Implementation of executable refinement for happening time points. Produces a strictly sorted sequence of all *is-htp*

```

fun https-exec :: plan  $\Rightarrow$  time list where
  https-exec [] = []
| https-exec ((tπ, Simple-Plan-Action n args) # π s) =
  insert-htp tπ (https-exec π s)
| https-exec ((tπ, Durative-Plan-Action n args d) # π s) =
  insert-htp tπ (insert-htp (tπ + d) (https-exec π s))

```

Basic properties of *https-exec*

lemma *https-exec-Nil*: $\text{https-exec } \pi s = [] \longleftrightarrow \pi s = []$
 ⟨proof⟩

lemma *https-exec-sorted*: $\text{sorted } (\text{https-exec } \pi s)$
 ⟨proof⟩

lemma *https-exec-distinct*: $\text{distinct } (\text{https-exec } \pi s)$
 ⟨proof⟩

Correctness proof for *https-exec*

Every time point in *https-exec* is a happening time point.

lemma *https-exec-is-htps*: $t_i \in \text{set } (\text{https-exec } \pi s) \implies \text{is-htp } \pi s t_i$
 ⟨proof⟩

lemma *https-exec-Cons-subset*: $\text{set } (\text{https-exec } \pi s) \subseteq \text{set } (\text{https-exec } (\pi \# \pi s))$
 ⟨proof⟩

lemma *https-exec-app-subset*: $\text{set } (\text{https-exec } \pi s_2) \subseteq \text{set } (\text{https-exec } (\pi s_1 @ \pi s_2))$
 ⟨proof⟩

Every happening time point is included in *https-exec*.

lemma *is-htps-https-exec*: $\text{is-htp } \pi s t_i \implies t_i \in \text{set } (\text{https-exec } \pi s)$
 ⟨proof⟩

lemma *is-htp-iff-https-exec*: $\forall t. \text{is-htp } \pi s t \longleftrightarrow t \in \text{set } (\text{https-exec } \pi s)$
 ⟨proof⟩

definition *consec-htps-in-interval* :: time list $\Rightarrow$ time $\Rightarrow$ time $\Rightarrow$ (time $\times$ time)
 list **where**
 consec-htps-in-interval htps t_i t_j =
 filter ($\lambda(t, t'). t_i \leq t \wedge t' \leq t_j$) (zip (butlast htps) (tl htps))

Construct all ground actions for a given plan action

```

fun simplify-action :: time list  $\Rightarrow$  (time  $\times$  plan-action)  $\Rightarrow$  (time  $\times$  ground-action)
list where
  simplify-action htps (t, Simple-Plan-Action n args) = (
    let a = the (resolve-action-schema n) in
    [(t, instantiate-action-schema a args)]
  )
| simplify-action htps (t, Durative-Plan-Action n args d) = (

```

let $a = \text{the } (\text{resolve-action-schema } n) \text{ in}$
 $(t, \text{inst-snap-action } a \text{ args } \text{At-Start}) \#$
 $(t+d, \text{inst-snap-action } a \text{ args } \text{At-End}) \#$
 $(\text{map } (\lambda(t_i, t_j).((t_i+t_j) / 2, \text{inst-snap-action } a \text{ args } \text{Over-All})) (\text{consec-https-in-interval}$
 $\text{https } t \text{ (} t+d \text{)))}$
 $)$

Auxiliary lemmas to prove properties about ' $\text{zip } (\text{butlast } xs) \text{ (tl } xs)$ ', which are needed to ultimately prove properties about *consec-https*.

lemma *zip-butlast-tl-hd*:

assumes *distinct* $(x_1 \# xs)$
and $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } (x_1 \# xs)) \text{ (tl } (x_1 \# xs)))$
shows $xs \neq [] \wedge x_2 = \text{hd } xs$
 $\langle \text{proof} \rangle$

lemma *zip-butlast-sublist*:

assumes *distinct* $(xs_1 @ x_1 \# xs_2)$
and $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } (xs_1 @ x_1 \# xs_2)) \text{ (tl } (xs_1 @ x_1 \# xs_2)))$
shows $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } (x_1 \# xs_2)) \text{ (tl } (x_1 \# xs_2)))$
 $\langle \text{proof} \rangle$

lemma *zip-butlast-tl-app1*:

assumes *distinct* xs
and $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) \text{ (tl } xs))$
shows $\exists xs_1 \ xs_2. xs = xs_1 @ x_1 \# x_2 \# xs_2$
 $\langle \text{proof} \rangle$

lemma *zip-butlast-tl-app2*:

$(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } (xs_1 @ x_1 \# x_2 \# xs_2)) \text{ (tl } (xs_1 @ x_1 \# x_2 \# xs_2)))$
 $\langle \text{proof} \rangle$

lemma *zip-butlast-tl-app*:

assumes *distinct* xs
shows $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) \text{ (tl } xs)) \longleftrightarrow (\exists xs_1 \ xs_2. xs = xs_1 @ x_1 \#$
 $x_2 \# xs_2)$
 $\langle \text{proof} \rangle$

lemma *sorted-distinct-zip-le*:

assumes *strict-sorted* xs
and $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) \text{ (tl } xs))$
shows $x_1 < x_2$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-zip-butlast-tl-leq-or-geq*:

assumes *strict-sorted* xs
and $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) \text{ (tl } xs))$
and $x \in \text{set } xs$
shows $x \leq x_1 \vee x_2 \leq x$
 $\langle \text{proof} \rangle$

lemma *consec-https-in-zip-butlast-tl*:
assumes $https = https_exec\ \pi s$
and $consec_https\ \pi s\ t_i\ t_j$
shows $(t_i, t_j) \in set\ (zip\ (butlast\ https)\ (tl\ https))$
 $\langle proof \rangle$

Executable refinement for *consec-https*.

lemma *happ-time-pts-exec-iff-consec-https*:
 $(t_i, t_j) \in set\ (zip\ (butlast\ (https_exec\ \pi s))\ (tl\ (https_exec\ \pi s))) \longleftrightarrow consec_https\ \pi s\ t_i\ t_j$
 $\langle proof \rangle$

Justification for *consec-https-in-interval*.

lemma *consec-https-in-interval-iff*:
assumes $t_i \leq t_j$
shows $\forall t\ t'.\ (consec_https\ \pi s\ t\ t' \wedge t_i \leq t \wedge t' \leq t_j) \longleftrightarrow (t, t') \in set\ (consec_https_in_interval\ (https_exec\ \pi s)\ t_i\ t_j)$
 $\langle proof \rangle$

All returned actions from *simplify-action* are correct instantiations of a *plan-action* for an induced simple plan.

lemma (in *wf-ast-problem*) *simp-act-inst-of-plan-act*:
assumes $wf_plan\ \pi s$
and $(t_\pi, \pi) \in set\ \pi s$
and $(t_a, a) \in set\ (simplify_action\ (https_exec\ \pi s)\ (t_\pi, \pi))$
shows $inst_of_plan_action\ \pi s\ (t_\pi, \pi)\ (t_a, a)$
 $\langle proof \rangle$

Function to insert a happening into a happening sequence.

fun *insort-happ* :: $happening \Rightarrow happening\ list \Rightarrow happening\ list$ **where**
 $insort_happ\ (t_i, A_i)\ [] = [(t_i, A_i)]$
 $| insort_happ\ (t_i, A_i)\ ((t_j, A_j) \# hs) =$
 $\quad (if\ t_i < t_j\ then\ (t_i, A_i) \# (t_j, A_j) \# hs$
 $\quad \quad else\ if\ t_i = t_j\ then\ (t_j, A_i\ @\ A_j) \# hs$
 $\quad \quad else\ (t_j, A_j) \# (insort_happ\ (t_i, A_i)\ hs))$

Properties of *insort-happ*

lemma *insort-happ-insort-insert*:
 $sorted\ (map\ fst\ hs) \implies insort_insert\ (fst\ h)\ (map\ fst\ hs) = map\ fst\ (insort_happ\ h\ hs)$
 $\langle proof \rangle$

insort-happ preserves sorted-ness.

lemma *insort-happ-distinct*: $strict_sorted\ (map\ fst\ hs) \implies distinct\ (map\ fst\ (insort_happ\ h\ hs))$
 $\langle proof \rangle$

lemma *insort-happ-not-Nil*:

assumes $A_i \neq []$ **and** $\forall (t_j, A_j) \in \text{set } hs. A_j \neq []$
shows $\forall (t_j, A_j) \in \text{set } (\text{insort-happ } (t_i, A_i) \text{ } hs). A_j \neq []$
 $\langle \text{proof} \rangle$

lemma *insort-happ-action-subset*:

$\forall (t_a, A) \in \text{set } hs \cup \{h\}. \exists A'. (t_a, A') \in \text{set } (\text{insort-happ } h \text{ } hs) \wedge \text{set } A \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *insort-happ2-action-subset*:

$\exists A'. (t_2, A') \in \text{set } (\text{insort-happ } (t_1, A_1) (\text{insort-happ } (t_2, A_2) \text{ } hs)) \wedge \text{set } A_2 \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *insort-happ-sorted*: $\text{sorted } (\text{map fst } hs) \implies \text{sorted } (\text{map fst } (\text{insort-happ } h \text{ } hs))$
 $\langle \text{proof} \rangle$

lemma *insort-happ-strict-sorted*:

$\text{strict-sorted } (\text{map fst } hs) \implies \text{strict-sorted } (\text{map fst } (\text{insort-happ } h \text{ } hs))$
 $\langle \text{proof} \rangle$

Function to insert multiple happening into a happening sequence.

fun *insort-mult-happs* :: *happening list* $\Rightarrow$ *happening list* $\Rightarrow$ *happening list* **where**
insort-mult-happs [] $hs_2 = hs_2$
 $| \text{insort-mult-happs } (h \# hs_1) \text{ } hs_2 = \text{insort-happ } h \text{ } (\text{insort-mult-happs } hs_1 \text{ } hs_2)$

fun *insort-timed-ground-acts* :: $(\text{time} \times \text{ground-action}) \text{ list} \Rightarrow \text{happening list} \Rightarrow$
happening list **where**

insort-timed-ground-acts [] $hs = hs$
 $| \text{insort-timed-ground-acts } ((t_a, a) \# as) \text{ } hs =$
 $\text{insort-happ } (t_a, [a]) \text{ } (\text{insort-timed-ground-acts } as \text{ } hs)$

lemma *insort-timed-ground-acts-correct*:

$\text{insort-mult-happs } (\text{map } (\lambda (t, a). (t, [a])) \text{ } as) \text{ } hs = \text{insort-timed-ground-acts } as \text{ } hs$
 $\langle \text{proof} \rangle$

Properties of *insort-mult-happs*

lemma *insort-mult-happs-strict-sorted*:

$\text{strict-sorted } (\text{map fst } hs_2) \implies \text{strict-sorted } (\text{map fst } (\text{insort-mult-happs } hs_1 \text{ } hs_2))$
 $\langle \text{proof} \rangle$

lemma *insort-mult-happs-not-Nil*:

assumes $\forall (t_a, A) \in \text{set } xs. A \neq []$ **and** $\forall (t_a, A) \in \text{set } ys. A \neq []$
shows $\forall (t_a, A) \in \text{set } (\text{insort-mult-happs } xs \text{ } ys). A \neq []$
 $\langle \text{proof} \rangle$

lemma *insort-mult-happs-action-subset1*:
 $(t_a, A) \in \text{set } hs_1 \implies \exists as'. (t_a, as') \in \text{set } (\text{insort-mult-happs } hs_1 \text{ } hs_2) \wedge \text{set } A \subseteq \text{set } as'$
 $\langle \text{proof} \rangle$

lemma *insort-mult-happs-action-subset2*:
assumes $(t_a, A) \in \text{set } hs_2$
shows $\exists A'. (t_a, A') \in \text{set } (\text{insort-mult-happs } hs_1 \text{ } hs_2) \wedge \text{set } A \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *insort-mult-happs-action-subset*:
 $\forall (t_a, A) \in \text{set } hs_1 \cup \text{set } hs_2. \exists A'. (t_a, A') \in \text{set } (\text{insort-mult-happs } hs_1 \text{ } hs_2) \wedge \text{set } A \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *insort-happ2-insort-mult-happs-action-subset-aux*:
assumes $(t_a, A) \in \text{set } hs_1$
shows $\exists A'. (t_a, A') \in \text{set } (\text{insort-happ } (t_1, A_1) (\text{insort-happ } (t_2, A_2) (\text{insort-mult-happs } hs_1 \text{ } hs_2))) \wedge \text{set } A \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *insort-happ2-insort-mult-happs-action-subset*:
 $\forall (t_a, A) \in \text{set } hs_1. \exists A'. (t_a, A') \in \text{set } (\text{insort-happ } (t_1, A_1) (\text{insort-happ } (t_2, A_2) (\text{insort-mult-happs } hs_1 \text{ } hs_2))) \wedge \text{set } A \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

We can define a function to construct a simplified plan for a given temporal plan.

fun *simplify-plan* :: *time list* $\Rightarrow$ *plan* $\Rightarrow$ *happening list* **where**
simplify-plan *htps* [] = []
| *simplify-plan* *htps* ($\pi \# \pi s$) =
insort-mult-happs (*map* ($\lambda(t_a, a). (t_a, [a])$) (*simplify-action* *htps* π)) (*simplify-plan* *htps* πs)

Definition of subset on happening sequences. This is used to simplify some proofs.

definition *happ-seq-member* :: *happening* $\Rightarrow$ *happening list* $\Rightarrow$ *bool* (**infix** $\in_{hs}$ 53) **where**
happ-seq-member *h* *hs* $\longleftrightarrow$ (*case* *h* of $(t_a, A) \Rightarrow \exists A_2. \text{set } A \subseteq \text{set } A_2 \wedge (t_a, A_2) \in \text{set } hs$)

definition *happ-seq-subset* :: *happening list* $\Rightarrow$ *happening list* $\Rightarrow$ *bool* (**infix** $\subseteq_{hs}$ 53) **where**
happ-seq-subset *hs*₁ *hs*₂ $\longleftrightarrow$ ($\forall h_1 \in \text{set } hs_1. h_1 \in_{hs} hs_2$)

Properties of ($\subseteq_{hs}$)

lemma *happ-seq-subset-Nil*: [] $\subseteq_{hs}$ *hs*

$\langle proof \rangle$

lemma *happ-seq-subset-refl*: $hs \subseteq_{hs} hs$
 $\langle proof \rangle$

lemma *happ-seq-subset-trans*: $hs_1 \subseteq_{hs} hs_2 \wedge hs_2 \subseteq_{hs} hs_3 \implies hs_1 \subseteq_{hs} hs_3$
 $\langle proof \rangle$

lemma *insort-happ-subset*: $hs \subseteq_{hs} insort-happ\ h\ hs$
 $\langle proof \rangle$

lemma *insort-mult-happs-subset*: $hs_1 \subseteq_{hs} insort-mult-happs\ hs_2\ hs_1$
 $\langle proof \rangle$

Happening sequence subset for simplify plan.

lemma *simplify-plan-happs-subset-Cons*: $simplify-plan\ htps\ \pi s \subseteq_{hs} simplify-plan\ htps\ (\pi \# \pi s)$
 $\langle proof \rangle$

lemma *simplify-plan-happs-subset-app*: $simplify-plan\ htps\ \pi s_2 \subseteq_{hs} simplify-plan\ htps\ (\pi s_1 @ \pi s_2)$
 $\langle proof \rangle$

Properties of *simplify-plan*

All happenings in a simplified plan are always non-*Nil*.

lemma *simp-plan-happ-not-Nil*:
fixes πs
defines $htps \equiv htps-exec\ \pi s$
shows $\forall (t_a, A) \in set\ (simplify-plan\ htps\ \pi s). A \neq []$
 $\langle proof \rangle$

A simplified plan produced by *simplify-plan* is strictly sorted.

lemma *simp-plan-strict-sorted*: $strict-sorted\ (map\ fst\ (simplify-plan\ htps\ \pi s))$
 $\langle proof \rangle$

A simplified plan contains all grounded actions for every *Simple-Plan-Action*.

lemma *simp-plan-corr-simp-acts*:
assumes $hs = simplify-plan\ htps\ \pi s$
and $(t_\pi, \pi) \in simple-acts\ \pi s$
shows $\exists A. (t_\pi, A) \in set\ hs \wedge the\ (res-inst\ \pi) \in set\ A$
 $\langle proof \rangle$

lemma *simp-plan-corr-dur-acts-start-aux*:
assumes $hs = simplify-plan\ htps\ \pi s$
and $(t_\pi, \pi) = hd\ \pi s$
and $(t_\pi, \pi) \in durative-acts\ \pi s$
shows $(\exists A. (t_\pi, A) \in set\ hs \wedge (the\ (res-inst-snap-action\ \pi\ At-Start)) \in set\ A)$
 $\langle proof \rangle$

A simplified plan contains all *At-Start*-snap-actions.

lemma *simp-plan-corr-dur-acts-start*:

fixes π

defines $a_{start} \equiv \text{the } (res-inst-snap-action \ \pi \ At-Start)$

assumes $hs = \text{simplify-plan } htps \ \pi s$

and $(t_\pi, \pi) \in \text{durative-acts } \pi s$

shows $\exists A. (t_\pi, A) \in \text{set } hs \wedge a_{start} \in \text{set } A$

<proof>

lemma *simp-plan-corr-dur-acts-end-aux*:

assumes $hs = \text{simplify-plan } htps \ \pi s$

and $(t_\pi, \pi) = \text{hd } \pi s$

and $(t_\pi, \pi) \in \text{durative-acts } \pi s$

shows $(\exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge (\text{the } (res-inst-snap-action \ \pi \ At-End)) \in \text{set } A)$

<proof>

A simplified plan contains all *At-End*-snap-actions.

lemma *simp-plan-corr-dur-acts-end*:

fixes π

defines $a_{end} \equiv \text{the } (res-inst-snap-action \ \pi \ At-End)$

assumes $hs = \text{simplify-plan } htps \ \pi s$

and $(t_\pi, \pi) \in \text{durative-acts } \pi s$

shows $\exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge a_{end} \in \text{set } A$

<proof>

lemma *member-map*: $a \in \text{set } A \implies f a \in \text{set } (\text{map } f A)$

<proof>

lemma *simp-plan-corr-dur-acts-overall-aux*:

assumes *strict-sorted* $htps$

and $hs = \text{simplify-plan } htps \ \pi s$

and $(t_\pi, \pi) = \text{hd } \pi s$

and $(t_\pi, \pi) \in \text{durative-acts } \pi s$

shows $\forall t_i \ t_j. (t_i, t_j) \in \text{set } (\text{consec-htps-in-interval } htps \ t_\pi \ (t_\pi + \text{duration } \pi))$

$\longrightarrow$

$(\exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge (\text{the } (res-inst-snap-action \ \pi \ Over-All)) \in \text{set } A)$

<proof>

A simplified plan contains all *Over-All*-snap-actions.

lemma (*in wf-fast-problem*) *simp-plan-corr-dur-acts-overall*:

fixes $\pi \ \pi s$

defines $htps \equiv \text{htps-exec } \pi s$

and $a_{inv} \equiv \text{the } (res-inst-snap-action \ \pi \ Over-All)$

assumes *wf-plan* πs

and $hs = \text{simplify-plan } htps \ \pi s$

and $(t_\pi, \pi) \in \text{durative-acts } \pi s$

shows $\forall t_i t_j. (\text{consec-https } \pi s \ t_i \ t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + \text{duration } \pi) \longrightarrow$
 $(\exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge a_{inv} \in \text{set } A)$

$\langle \text{proof} \rangle$

Flatten a happening into individual timed grounded actions. This is used to simplify proofs.

fun *set-happ-acts* :: *happening list* $\Rightarrow$ (*time* $\times$ *ground-action*) *set* **where**
set-happ-acts [] = {}
| *set-happ-acts* ((*t_a*, *A*) # *hs*) = {(*t_a*', *a*'). *t_a* = *t_a*' $\wedge$ *a*' $\in$ *set A*} $\cup$ *set-happ-acts hs*

lemma *set-happ-acts-refine*:

$(\exists A. (t_a, A) \in \text{set } hs \wedge a \in \text{set } A) \longleftrightarrow ((t_a, a) \in \text{set-happ-acts } hs)$

$\langle \text{proof} \rangle$

lemma *set-happ-acts-insort-happ*:

set-happ-acts (*insort-happ h hs*) = *set-happ-acts [h]* $\cup$ *set-happ-acts hs*

$\langle \text{proof} \rangle$

lemma *set-happ-acts-insort-mult-happs*:

set-happ-acts (*insort-mult-happs hs₁ hs₂*) = *set-happ-acts hs₁* $\cup$ *set-happ-acts*
hs₂

$\langle \text{proof} \rangle$

lemma *set-happ-acts-to-happ*: *set-happ-acts* (*map* ($\lambda(t_a, a). (t_a, [a])$) *xs*) = *set xs*

$\langle \text{proof} \rangle$

lemma *simp-plan-corr-inst-plan-act-aux*:

assumes *strict-sorted https*

and *hs* = *simplify-plan https* πs

shows $\forall (t_a, a) \in \text{set-happ-acts } hs. \exists (t_\pi, \pi) \in \text{set } \pi s. (t_a, a) \in \text{set } (\text{simplify-action}$
https (*t_π*, π))

$\langle \text{proof} \rangle$

Every ground action in a simplified plan is the instantiation of a plan action.

lemma (**in** *wf-ast-problem*) *simp-plan-corr-inst-plan-act*:

fixes πs

defines *https* $\equiv$ *https-exec* πs

assumes *wf-plan* πs

and *hs* = *simplify-plan https* πs

and (*t_a*, *A*) $\in$ *set hs*

and *a* $\in$ *set A*

shows $\exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s \ (t_\pi, \pi) \ (t_a, a)$

$\langle \text{proof} \rangle$

Finally the correctness proof for *simplify-plan*

lemma (**in** *wf-ast-problem*) *simplify-plan-correct*:

fixes πs

defines *https* $\equiv$ *https-exec* πs

```

assumes wf-plan  $\pi s$ 
shows ind-happ-seq  $\pi s$  (simplify-plan https  $\pi s$ )
<proof>

```

```

lemma (in wf-ast-problem)
assumes wf-plan  $\pi s$ 
shows ind-happ-seq  $\pi s$  (simplify-plan (https-exec  $\pi s$ )  $\pi s$ )
<proof>

```

With the correctness proof of *simplify-plan*. We can easily proof that for every plan there exists an induced happening sequence.

```

lemma (in wf-ast-problem) ind-happ-seq-exists:
assumes wf-plan  $\pi s$ 
shows  $\exists hs.$  ind-happ-seq  $\pi s$   $hs$ 
<proof>

```

```

lemma (in wf-ast-problem) simplify-plan-wf-ground-action-aux:
fixes  $\pi s$ 
defines https  $\equiv$  https-exec  $\pi s$ 
assumes wf-plan  $\pi s$ 
and  $hs =$  simplify-plan https  $\pi s$ 
and  $(t_a, A) \in$  set  $hs$ 
and  $a \in$  set  $A$ 
shows wf-ground-action  $a$ 
<proof>

```

```

lemma (in wf-ast-problem) simplify-plan-wf-ground-action:
fixes  $\pi s$ 
defines https  $\equiv$  https-exec  $\pi s$ 
assumes wf-plan  $\pi s$ 
and  $hs =$  simplify-plan https  $\pi s$ 
shows  $\forall (t_a, A) \in$  set  $hs. \forall a \in$  set  $A. wf-ground-action a$ 
<proof>

```

Recursive refinement for *valid-happ-seq*.

```

fun valid-happ-seq-from :: world-model  $\Rightarrow$  happening list  $\Rightarrow$  bool where
  valid-happ-seq-from  $s$  []  $\longleftrightarrow$  close-world  $s \models$  (goal  $P$ )
| valid-happ-seq-from  $s$  ( $h \# hs$ )  $\longleftrightarrow$ 
  (happ-enabled  $h$   $s \wedge$  valid-happ-seq-from (apply-happ-exec  $h$   $s$ )  $hs$ )

```

Justification of the refinement for *valid-happ-seq*.

```

lemma valid-happ-seq-from-refine:
assumes wf-happ-seq  $hs$ 
shows valid-happ-seq-from  $M$   $hs \longleftrightarrow (\exists M'. \text{valid-happ-seq } M \text{ } hs \text{ } M' \wedge M' \text{ }^c \models$ 
(goal  $P$ ))
<proof>

```

```

definition mp-Fevl :: (object, rat) mapping where

```

$mp\text{-}Fevl = Mapping.of\text{-}alist \text{ (fold } (\lambda a \text{ fev. case } a \text{ of } Atom \text{ (eqAtm } f \text{ (TimeEnt } t)) \Rightarrow (f,t)\#fev \mid - \Rightarrow fev) \text{ (init } P) \text{ []})}$

lemma *mp-func-eval-correct[simp]: Mapping.lookup mp-Fevl = func-eval*
<proof>

fun *duration-matches'* :: (object, rat) mapping $\Rightarrow$ time $\Rightarrow$ term duration-constraint
 $\Rightarrow$ (variable $\times$ type) list $\Rightarrow$ object list $\Rightarrow$ bool **where**
duration-matches' mp-fe d No-Const params args $\longleftrightarrow d \geq 0$
 $\mid$ *duration-matches'* mp-fe d (Time-Const op d') params args $\longleftrightarrow$
 (case op of
 LEQ $\Rightarrow d \leq d'$
 $\mid$ EQ $\Rightarrow d = d'$
 $\mid$ GEQ $\Rightarrow d \geq d'$)
 $\mid$ *duration-matches'* mp-fe d (Func-Const op f vs) params args $\longleftrightarrow$
 (let tsubst = subst-term (the o (map-of (zip (map fst params) args))) in
 case Mapping.lookup mp-fe (FuncEnt f (map tsubst vs)) of
 None $\Rightarrow$ False
 $\mid$ Some d' $\Rightarrow$ (case op of
 LEQ $\Rightarrow d \leq d'$
 $\mid$ EQ $\Rightarrow d = d'$
 $\mid$ GEQ $\Rightarrow d \geq d'$))

lemma (in wf-ast-problem) *duration-matches'-correct:*
duration-matches' mp-Fevl d dc params args $\longleftrightarrow$ duration-matches d dc params
args
<proof>

fun *consec-htps-in-interval-exec* :: time list $\Rightarrow$ time $\Rightarrow$ time $\Rightarrow$ (time $\times$ time) list
where
consec-htps-in-interval-exec (t#t'#ts) t_i t_j =
 (if $t < t_i$ then *consec-htps-in-interval-exec* (t'#ts) t_i t_j
 else if $t_i \leq t \wedge t' \leq t_j$ then (t,t')#(*consec-htps-in-interval-exec* (t'#ts) t_i t_j)
 else [])
 $\mid$ *consec-htps-in-interval-exec* ts t_i t_j = []

lemma *consec-htps-in-interval-exec-correct:*
assumes *strict-sorted ts*
shows *consec-htps-in-interval ts t_i t_j = consec-htps-in-interval-exec ts t_i t_j*
<proof>

fun *place-inv-snap-acts* :: time list $\Rightarrow$ time $\Rightarrow$ time $\Rightarrow$ ground-action $\Rightarrow$ (time $\times$
 ground-action) list **where**
place-inv-snap-acts (t#t'#ts) t_i t_j a_{inv} =
 (if $t < t_i$ then *place-inv-snap-acts* (t'#ts) t_i t_j a_{inv}
 else if $t_i \leq t \wedge t' \leq t_j$ then ((t+t') div 2, a_{inv})#(*place-inv-snap-acts* (t'#ts) t_i
 t_j a_{inv})
 else [])
 $\mid$ *place-inv-snap-acts* ts t_i t_j a_{inv} = []

lemma *place-inv-snap-acts-correct*:
assumes *strict-sorted ts*
shows *place-inv-snap-acts ts t₁ t₂ a_{inv} = map (λ(t_i,t_j).((t_i+t_j) div 2,a_{inv}))*
(consec-https-in-interval ts t₁ t₂)
 ⟨*proof*⟩

Lift *simplify-action* into an Error Monad.

fun *simplify-actionE* :: - ⇒ (object, type) mapping ⇒ (object, rat) mapping ⇒
time list ⇒ (time × plan-action) ⇒ -(time × ground-action) list **where**
simplify-actionE stg mp mp-fe https (t_π,Simple-Plan-Action n args) = do {
check (t_π ≥ 0) (ERRS "Invalid starting time point");
a ← resolve-action-schemaE n;
check (case a of Simple-Action-Schema - - - ⇒ True | - ⇒ False) (ERRS
"Invalid Plan action type");
check (action-params-match2 stg mp a args) (ERRS "Parameter mismatch");
Error-Monad.return [(t_π,instantiate-action-schema a args)]
}
| simplify-actionE stg mp mp-fe https (t_π,Durative-Plan-Action n args d) = do {
check (t_π ≥ 0) (ERRS "Invalid starting time point");
a ← resolve-action-schemaE n;
check (case a of Durative-Action-Schema - - - - ⇒ True | - ⇒ False) (ERRS
"Invalid Plan action type");
check (action-params-match2 stg mp a args) (ERRS "Parameter mismatch");
check (d ≥ 0) (ERRS "Invalid duration (negative)");
check (duration-matches' mp-fe d (duration-constraint a) (parameters a) args)
(ERRS "Duration constraint not satisfied");
let a_{inv} = inst-snap-action a args Over-All in
Error-Monad.return (
(t_π,inst-snap-action a args At-Start) #
(t_π+d,inst-snap-action a args At-End) #
(place-inv-snap-acts https t_π (t_π+d) a_{inv})
)
}

Justification of refinement for *simplify-action*.

lemma (in *wf-ast-problem*) *simplify-actionE-return-iff*:
assumes *wf-domain* **and** *strict-sorted https*
shows *simplify-actionE STG mp-objT mp-Fevl https (t_π,π) = Inr as*
 ⇔ (*wf-plan-action π ∧ t_π ≥ 0 ∧ simplify-action https (t_π,π) = as*)
 ⟨*proof*⟩

Lift *simplify-plan* into an Error Monad.

fun *simplify-planE* :: - ⇒ (object, type) mapping ⇒ (object, rat) mapping ⇒ *time*
list ⇒ *plan* ⇒ -+happening list **where**
simplify-planE stg mp mp-fe https [] = do {
Error-Monad.return []
}

```

| simplify-planE stg mp mp-fe https ( $\pi \# \pi s$ ) = do {
  as  $\leftarrow$  simplify-actionE stg mp mp-fe https  $\pi$ ;
  hs  $\leftarrow$  simplify-planE stg mp mp-fe https  $\pi s$ ;
  Error-Monad.return (insort-timed-ground-acts as hs)
}

```

Justification of refinement for *simplify-plan*

lemma (in *wf-ast-problem*) *simplify-planE-return-iff*[*return-iff*]:
assumes *wf-domain* **and** *strict-sorted https*
shows *simplify-planE STG mp-objT mp-Fevl https πs = Inr hs*
 $\longleftrightarrow$ (*simplify-plan https πs = hs $\wedge$ wf-plan πs)*

<proof>

type-synonym 'a tree-ht = ('a $\times$ nat) tree

fun *avl-height* :: *happening tree-ht* $\Rightarrow$ nat **where**
avl-height $\langle \rangle$ = 0
| *avl-height* $\langle l, ((t, as), ht), r \rangle$ = ht

definition *avl-node* :: *happening tree-ht* $\Rightarrow$ *happening* $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**
avl-node l h r = $\langle l, (h, (\max (\text{avl-height } l) (\text{avl-height } r) + 1)), r \rangle$

definition *avl-balL* :: *happening tree-ht* $\Rightarrow$ *happening* $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**
avl-balL l a r =
(if *avl-height* l = *avl-height* r + 2 then
case l of $\langle bl, (b, -), br \rangle \Rightarrow$
if *avl-height* bl < *avl-height* br then
case br of $\langle cl, (c, -), cr \rangle \Rightarrow$ *avl-node* (*avl-node* bl b cl) c (*avl-node* cr a r)
else *avl-node* bl b (*avl-node* br a r)
else *avl-node* l a r)

definition *avl-balR* :: *happening tree-ht* $\Rightarrow$ *happening* $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**
avl-balR l a r =
(if *avl-height* r = *avl-height* l + 2 then
case r of $\langle bl, (b, -), br \rangle \Rightarrow$
if *avl-height* bl > *avl-height* br then
case bl of $\langle cl, (c, -), cr \rangle \Rightarrow$ *avl-node* (*avl-node* l a cl) c (*avl-node* cr b br)
else *avl-node* (*avl-node* l a bl) b br
else *avl-node* l a r)

fun *insert-happ-to-tree* :: *happening* $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**
insert-happ-to-tree (t_i, A_i) $\langle \rangle$ = *avl-node* $\langle \rangle$ (t_i, A_i) $\langle \rangle$
| *insert-happ-to-tree* (t_i, A_i) $\langle l, ((t_j, A_j), h), r \rangle$ = (
if $t_i < t_j$ then *avl-balL* (*insert-happ-to-tree* (t_i, A_i) l) (t_j, A_j) r

else if $t_i = t_j$ then *avl-node* $l (t_j, A_i @ A_j) r$
 else *avl-balR* $l (t_j, A_j) (\text{insert-happ-to-tree } (t_i, A_i) r)$

fun *insert-timed-ground-acts-to-tree* :: (*time* $\times$ *ground-action*) *list* $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**
insert-timed-ground-acts-to-tree [] *htree* = *htree*
 | *insert-timed-ground-acts-to-tree* $((t_a, a) \# as)$ *htree* =
insert-happ-to-tree $(t_a, [a]) (\text{insert-timed-ground-acts-to-tree } as \text{ htree})$

definition *avl-inorder* :: *happening tree-ht* $\Rightarrow$ *happening list* **where**
avl-inorder *htree* = *map fst (inorder htree)*

fun *simplify-planE-avl'* :: - $\Rightarrow$ (*object, type*) *mapping* $\Rightarrow$ (*object, rat*) *mapping* $\Rightarrow$ *time list* $\Rightarrow$ *plan* $\Rightarrow$ -+*happening tree-ht* **where**
simplify-planE-avl' *G mp mp-fe htps* [] = *do* {
Error-Monad.return $\langle \rangle$
 }
 | *simplify-planE-avl'* *G mp mp-fe htps* $(\pi \# \pi s)$ = *do* {
as $\leftarrow$ *simplify-actionE* *G mp mp-fe htps* π ;
htree $\leftarrow$ *simplify-planE-avl'* *G mp mp-fe htps* πs ;
Error-Monad.return $(\text{insert-timed-ground-acts-to-tree } as \text{ htree})$
 }

fun *simplify-planE-avl* :: - $\Rightarrow$ (*object, type*) *mapping* $\Rightarrow$ (*object, rat*) *mapping* $\Rightarrow$ *time list* $\Rightarrow$ *plan* $\Rightarrow$ -+*happening list* **where**
simplify-planE-avl *G mp mp-fe htps* πs = *do* {
htree $\leftarrow$ *simplify-planE-avl'* *G mp mp-fe htps* πs ;
Error-Monad.return $(\text{avl-inorder } htree)$
 }

lemma *insort-happ-consL*:

assumes $\forall (t_j, A_j) \in \text{set } hs_1. t_j < t_i$
shows $hs_1 @ (\text{insort-happ } (t_i, A_i) hs_2) = \text{insort-happ } (t_i, A_i) (hs_1 @ hs_2)$
 $\langle \text{proof} \rangle$

lemma *insort-happ-consR*:

assumes $\forall (t_j, A_j) \in \text{set } hs_2. t_i < t_j$
shows $(\text{insort-happ } (t_i, A_i) hs_1) @ hs_2 = \text{insort-happ } (t_i, A_i) (hs_1 @ hs_2)$
 $\langle \text{proof} \rangle$

lemma *inorder-avl-balL*: *avl-inorder* $(\text{avl-balL } l a r) = \text{avl-inorder } l @ a \#$
avl-inorder *r*
 $\langle \text{proof} \rangle$

lemma *inorder-avl-balR*: *avl-inorder* $(\text{avl-balR } l a r) = \text{avl-inorder } l @ a \#$
avl-inorder *r*
 $\langle \text{proof} \rangle$

lemma *strict-sorted-appl*: $\text{strict-sorted } (a @ [y]) \implies \forall x \in \text{set } a. x < y$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-consg*: $\text{strict-sorted } (y \# a) \implies \forall x \in \text{set } a. y < x$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-avl-inorder-l*:
assumes $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } \langle l, ((t_i, A_i), h), r \rangle))$
shows $\forall (t_j, A_j) \in \text{set } (\text{avl-inorder } l). t_j < t_i$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-avl-inorder-r*:
assumes $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } \langle l, ((t_i, A_i), h), r \rangle))$
shows $\forall (t_j, A_j) \in \text{set } (\text{avl-inorder } r). t_i < t_j$
 $\langle \text{proof} \rangle$

lemma *insert-happ-to-tree-set*:
assumes $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } \text{htree}))$
shows $\text{set } (\text{map fst } (\text{avl-inorder } (\text{insert-happ-to-tree } (t_i, A_i) \text{ htree}))) = \text{set } (\text{map fst } (\text{avl-inorder } \text{htree})) \cup \{t_i\}$
 $\langle \text{proof} \rangle$

lemma *insert-timed-ground-act-to-tree-strict-sorted*:
assumes $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } \text{htree}))$
shows $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } (\text{insert-happ-to-tree } h \text{ htree})))$
 $\langle \text{proof} \rangle$

lemma *insert-timed-ground-acts-to-tree-strict-sorted*:
assumes $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } \text{htree}))$
shows $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } (\text{insert-timed-ground-acts-to-tree as htree})))$
 $\langle \text{proof} \rangle$

lemma *insert-happ-to-tree-equiv*:
assumes $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } \text{htree}))$
shows $\text{avl-inorder } (\text{insert-happ-to-tree } (t_i, A_i) \text{ htree}) = \text{insort-happ } (t_i, A_i) (\text{avl-inorder } \text{htree})$
 $\langle \text{proof} \rangle$

lemma *insert-timed-ground-acts-to-tree-equiv*:
assumes $\text{strict-sorted } (\text{map fst } (\text{avl-inorder } \text{htree}))$
shows $\text{insort-timed-ground-acts as } (\text{avl-inorder } \text{htree}) = \text{avl-inorder } (\text{insert-timed-ground-acts-to-tree as htree})$
 $\langle \text{proof} \rangle$

lemma *simplify-planE-avl'-strict-sorted*:
assumes $\text{simplify-planE-avl'} \text{ STG mp-objT mp-Fevl htps } \pi s = \text{Inr htree}$

shows *strict-sorted* (*map fst (avl-inorder htree)*)
 ⟨*proof*⟩

lemma *simplify-planE-avl'-equiv*:

assumes *simplify-planE STG mp-objT mp-Fevl https πs = Inr hs*
and *simplify-planE-avl' STG mp-objT mp-Fevl https πs = Inr htree*
shows *hs = avl-inorder htree*
 ⟨*proof*⟩

lemma *simplify-planE-avl-equiv-Inr*:

shows ($\exists hs. \textit{simplify-planE STG mp-objT mp-Fevl https } \pi s = \textit{Inr hs}$)
 $\longleftrightarrow$ ($\exists htree. \textit{simplify-planE-avl' STG mp-objT mp-Fevl https } \pi s = \textit{Inr htree}$)
 ⟨*proof*⟩

lemma *simplify-planE-avl-equiv*:

shows *simplify-planE STG mp-objT mp-Fevl https πs = Inr hs*
 $\longleftrightarrow$ *simplify-planE-avl STG mp-objT mp-Fevl https πs = Inr hs*
 ⟨*proof*⟩

lemma (**in** *wf-ast-problem*)

assumes *wf-domain*
and *simplify-planE-avl STG mp-objT mp-Fevl (https-exec πs) πs = Inr hs*
shows *ind-happ-seq πs hs and wf-plan πs*
 ⟨*proof*⟩

Lift *valid-happ-seq-from* into an Error Monad, and use efficient happening execution *en-exE*

fun *valid-happ-seq-fromE* :: *nat* $\Rightarrow$ *world-model* $\Rightarrow$ *happening list* $\Rightarrow$ *-+unit*
where
valid-happ-seq-fromE si s [] =
check (holds s (goal P)) (ERRS "Postcondition does not hold")
| *valid-happ-seq-fromE si s (h#hs)* = *do* {
s $\leftarrow$ *en-exE h s* $<+?$ ($\lambda e \dots$ *shows "at step " o shows si o shows ": " o e ()*);
valid-happ-seq-fromE (si+1) s hs
}

For the refinement, we need to show that the world models only contain atoms, i.e., containing only atoms is an invariant under execution of well-formed happenings.

lemma (**in** *wf-ast-problem*) *wf-happ-only-add-atoms*:

assumes *wm-basic s*
and $\forall a \in \textit{set as. wf-ground-action a}$
shows *wm-basic (apply-happ-exec (t,as) s)*
 ⟨*proof*⟩

lemma (**in** *wf-ast-problem*) *wf-happ-only-add-atoms'*:

assumes *wf-world-model s*

and $\forall a \in \text{set } as. \text{wf-ground-action } a$
shows $\text{wf-world-model } (\text{apply-happ-exec } (t, as) \ s)$
 $\langle \text{proof} \rangle$

Justification for refinement of *valid-happ-seq-from*.

lemma (**in** *wf-ast-problem*) *valid-happ-seq-fromE-return-iff*[*return-iff*]:
assumes *wm-basic* *s*
and $\forall (t, as) \in \text{set } hs. \forall a \in \text{set } as. \text{wf-ground-action } a$
shows $\text{valid-happ-seq-from } s \ hs \longleftrightarrow \text{valid-happ-seq-fromE } k \ s \ hs = \text{Inr } ()$
 $\langle \text{proof} \rangle$

Now we can define a refinement for *valid-plan-from2*.

fun *valid-plan-from2-exec* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *bool* **where**
valid-plan-from2-exec *s* $\pi s \longleftrightarrow$
 $(\text{wf-plan } \pi s \wedge \text{valid-happ-seq-from } s \ (\text{simplify-plan } (\text{https-exec } \pi s) \ \pi s))$

With $\llbracket \text{wf-ast-problem } ?P; \text{ast-problem.ind-happ-seq } ?P \ ?\pi s \ ?hs; \text{ast-problem.ind-happ-seq } ?P \ ?\pi s \ ?hs'; \text{ast-problem.wf-plan } ?P \ ?\pi s; \text{valid-happ-seq } ?M_0 \ ?hs \ ?M_n \rrbracket \implies \text{valid-happ-seq } ?M_0 \ ?hs' \ ?M_n$ we can prove, that if there exists a valid induced happening sequence, then simplify plan is also valid.

lemma (**in** *wf-ast-problem*) *simplify-plan-valid-exists1*:
fixes πs
defines $\text{https} \equiv \text{https-exec } \pi s$
assumes *wf-plan* πs
and $\exists hs. \text{ind-happ-seq } \pi s \ hs \wedge \text{valid-happ-seq } M \ hs \ M' \wedge M' \ c \models \text{(goal } P)$
shows $\text{valid-happ-seq } M \ (\text{simplify-plan } \text{https } \pi s) \ M' \wedge M' \ c \models \text{(goal } P)$
 $\langle \text{proof} \rangle$

Other way round is easy, since *simplify-plan* is an induced happening sequence.

lemma (**in** *wf-ast-problem*) *simplify-plan-valid-exists2*:
fixes πs
defines $\text{https} \equiv \text{https-exec } \pi s$
assumes *wf-plan* πs
and $\text{valid-happ-seq } M \ (\text{simplify-plan } \text{https } \pi s) \ M' \wedge M' \ c \models \text{(goal } P)$
shows $\exists hs. \text{ind-happ-seq } \pi s \ hs \wedge \text{valid-happ-seq } M \ hs \ M' \wedge M' \ c \models \text{(goal } P)$
 $\langle \text{proof} \rangle$

Justification for the refinement of *valid-plan-from2*.

lemma (**in** *wf-ast-problem*) *valid-plan-from2-exec-refine*:
 $\text{valid-plan-from2 } s \ \pi s \longleftrightarrow \text{valid-plan-from2-exec } s \ \pi s$
 $\langle \text{proof} \rangle$

Now we can also lift *valid-plan-from2-exec* into an Error Monad, using *simplify-planE* and *valid-happ-seq-fromE*

fun *valid-plan-from2E* :: $- \Rightarrow (\text{object}, \text{type}) \text{ mapping} \Rightarrow (\text{object}, \text{rat}) \text{ mapping} \Rightarrow$
 $\text{world-model} \Rightarrow \text{plan} \Rightarrow \text{+unit}$ **where**

```

valid-plan-from2E stg mp mp-fe s  $\pi s$  = do {
  hs  $\leftarrow$  simplify-planE-avl stg mp mp-fe (https-exec  $\pi s$ )  $\pi s$ ;
  valid-happ-seq-fromE 1 s hs
}

```

lemma (in *wf-ast-problem*) *valid-plan-fromE-wf-plan*:
assumes *valid-plan-from2E STG mp-objT mp-Fevl s πs = Inr ()*
shows *wf-plan πs*
<proof>

Refinement lemma for our plan checking algorithm

lemma (in *wf-ast-problem*) *valid-plan-from2E-return-iff*:
assumes *wf-world-model s*
shows *valid-plan-from2E STG mp-objT mp-Fevl s πs = Inr () $\longleftrightarrow$ valid-plan-from2E s πs*
<proof>

lemma (in *wf-ast-problem*)
assumes *wf-world-model s*
shows *valid-plan-from2E STG mp-objT mp-Fevl s πs = Inr () $\longleftrightarrow$ valid-plan-from2E s πs*
<proof>

lemmas *valid-plan-from2E-return-iff'[return-iff]*
= *wf-ast-problem.valid-plan-from2E-return-iff[of P, OF wf-ast-problem.intro]*

end — Context of *ast-problem*

2.3 Executable Plan Checker

We obtain the main plan checker by combining the well-formedness check and executability check.

definition *check-all-list P l msg msgf* $\equiv$
forallM (λx . check (P x) ($\lambda :: \text{unit}$. shows msg o shows "': " o msgf x)) l <+?
snd

lemma *check-all-list-return-iff'[return-iff]*: *check-all-list P l msg msgf = Inr () $\longleftrightarrow$ ($\forall x \in \text{set } l$. P x)*
<proof>

definition *check-wf-types* $D \equiv do \{$
 check-all-list $(\lambda(-,t). t = "object" \vee t \in fst' set (types D)) (types D) \text{ "Undeclared$
 *supertype" (shows o snd)}
 $\}$*

lemma *check-wf-types-return-iff*[*return-iff*]: *check-wf-types* $D = Inr () \longleftrightarrow ast_domain.wf_types$
 D
<proof>

definition *check-wf-domain* $D stg conT \equiv do \{$
 check-wf-types D ;
 check $(distinct (map (predicate-decl.pred) (predicates D))) (ERRS \text{ "Duplicate$
 *predicate declaration"});
 check-all-list $(ast_domain.wf_predicate_decl D) (predicates D) \text{ "Malformed predi-$
 *cate declaration" (shows o predicate.name o predicate-decl.pred)};
 check $(distinct (map (function-decl.func) (functions D))) (ERRS \text{ "Duplicate func-$
 *tion declaration"});
 check-all-list $(ast_domain.wf_function_decl D) (functions D) \text{ "Malformed function$
 *declaration" (shows o func.name o function-decl.func)};
 check $(distinct (map fst (consts D))) (ERRS \text{ "Duplicate constant declaration"}$);
 check $(\forall (n, T) \in set (consts D). ast_domain.wf_type D T) (ERRS \text{ "Malformed$
 *type"});
 check $(distinct (map ast_action_schema.name (actions D))) (ERRS \text{ "Duplicate$
 *action name"});
 check-all-list $(ast_domain.wf_action_schema' D stg conT) (actions D) \text{ "Malformed$
 *action" (shows o ast_action_schema.name)}
 $\}$*******

lemma *check-wf-domain-return-iff*[*return-iff*]:
check-wf-domain $D stg conT = Inr () \longleftrightarrow ast_domain.wf_domain' D stg conT$
<proof>

definition *prepend-err-msg* $msg e \equiv \lambda::unit. shows msg o shows "': " o e ()$

definition *check-wf-problem* $P stg conT mp \equiv do \{$
 let $D = ast_problem.domain P$;
 check-wf-domain $D stg conT <+ ? prepend_err_msg \text{ "Domain not well-formed"}$;
 check $(distinct (map fst (objects P) @ map fst (consts D))) (ERRS \text{ "Duplicate$
 *object declaration"});
 check $((\forall (n, T) \in set (objects P). ast_domain.wf_type D T)) (ERRS \text{ "Malformed$
 *type"});
 check $(distinct (init P)) (ERRS \text{ "Duplicate fact in initial state"}$);
 check $(\forall f \in set (init P). ast_problem.wf_fmla_atom2' P mp stg f \vee ast_problem.wf_func_assign'$
 *P mp stg f) (ERRS \text{ "Malformed formula in initial state"});
 check $(ast_domain.wf_fmla' D (Mapping.lookup mp) stg (goal P)) (ERRS \text{ "Malformed$
 *goal formula"})
 $\}$****

lemma *check-wf-problem-return-iff*[*return-iff*]:
 $check-wf-problem\ P\ stg\ conT\ mp = Inr\ () \longleftrightarrow ast-problem.wf-problem'\ P\ stg\ conT\ mp$
<proof>

definition *check-plan* $P\ \pi s \equiv do\ \{\$
 $let\ stg = ast-domain.STG\ (ast-problem.domain\ P);$
 $let\ conT = ast-domain.mp-constT\ (ast-problem.domain\ P);$
 $let\ mp = ast-problem.mp-objT\ P;$
 $let\ mp-fe = ast-problem.mp-Fevl\ P;$
 $check-wf-problem\ P\ stg\ conT\ mp;$
 $ast-problem.valid-plan-from2E\ P\ stg\ mp\ mp-fe\ (ast-problem.I\ P)\ \pi s$
 $\} <+?\ (\lambda e.\ String.implode\ (e\ ()\ '''))$

Correctness theorem of the plan checker: It returns *Inr ()* if and only if the problem is well-formed and the plan is valid.

theorem *check-plan-return-iff*[*return-iff*]:
 $check-plan\ P\ \pi s = Inr\ () \longleftrightarrow ast-problem.wf-problem\ P \wedge ast-problem.valid-plan2\ P\ \pi s$
<proof>

find-theorems *ast-problem.valid-plan-from2*

2.4 Code Setup

In this section, we set up the code generator to generate verified code for our plan checker.

2.4.1 Code Equations

We first register the code equations for the functions of the checker. Note that we not necessarily register the original code equations, but also optimized ones.

lemmas *wf-domain-code* =
 $ast-domain.sig-def$
 $ast-domain.wf-types-def$
 $ast-domain.wf-type.simps$
 $ast-domain.wf-predicate-decl.simps$
 $ast-domain.STG-def$
 $ast-domain.is-of-type'-def$
 $ast-domain.wf-atom'.simps$
 $ast-domain.wf-pred-atom'.simps$
 $ast-domain.wf-fmla'.simps$
 $ast-domain.wf-fmla-atom1'.simps$
 $ast-domain.wf-effect'.simps$
 $ast-domain.wf-action-schema'.simps$
 $ast-domain.wf-domain'-def$

ast-domain.subst-term.simps
ast-domain.mp-constT-def

ast-domain.wf-function-decl.simps
ast-domain.func-sig-def
ast-domain.wf-func-args'.simps
ast-domain.wf-duration-const'.simps
ast-domain.acts-non-intrf-def

declare *wf-domain-code*[*code*]

lemmas *wf-problem-code* =
ast-problem.wf-problem'-def
ast-problem.wf-fact'-def
ast-problem.is-obj-of-type-alt
ast-problem.wf-fact-def
ast-problem.wf-plan-action.simps
ast-domain.subtype-edge.simps

declare *wf-problem-code*[*code*]

lemmas *check-code* =
ast-problem.valid-plan2-def
ast-problem.valid-plan-from2E.simps
ast-problem.res-inst.simps
ast-domain.resolve-action-schema-def
ast-domain.resolve-action-schemaE-def
ast-problem.I-def
ast-domain.instantiate-action-schema.simps
ast-problem.holds-def
ast-problem.mp-objT-def
ast-problem.is-obj-of-type-impl-def
ast-problem.wf-fmla-atom2'-def
valuation-def

ast-problem.valid-happ-seq-fromE.simps
ast-problem.htps-exec.simps
ast-problem.insort-htp.simps
ast-problem.simplify-planE.simps
ast-problem.simplify-actionE.simps
ast-problem.insort-happ.simps
ast-domain.inst-snap-action.simps
ast-domain.filter-time-spec-def
ast-domain.conjunct-effects-def

ast-problem.apply-happ-exec.simps
ast-problem.en-exE-def
ast-problem.insort-mult-happs.simps

```

ast-problem.duration-matches'.simps
ast-problem.wf-func-assign'.simps

ast-problem.mp-Fevl-def
ast-problem.place-inv-snap-acts.simps

ast-problem.insort-timed-ground-acts.simps
ast-problem.insert-happ-to-tree.simps
ast-problem.insert-timed-ground-acts-to-tree.simps
ast-problem.avl-inorder-def
ast-problem.simplify-planE-avl.simps
ast-problem.simplify-planE-avl'.simps
ast-problem.avl-height.simps
ast-problem.avl-node-def
ast-problem.avl-balR-def
ast-problem.avl-balL-def

```

```
declare check-code[code]
```

2.4.2 Setup for Containers Framework

```
derive (linorder) compare rat
```

```

derive (eq) ceq predicate func atom object formula
derive ccompare predicate func atom object formula
derive (no) cenum atom object formula
derive (rbt) set-impl func atom object formula

```

```
derive (rbt) mapping-impl object
```

```
derive linorder predicate func object atom object atom formula
```

2.4.3 More Efficient Distinctness Check for Linorders

```

fun no-stutter :: 'a list ⇒ bool where
  no-stutter [] = True
| no-stutter [-] = True
| no-stutter (a#b#l) = (a≠b ∧ no-stutter (b#l))

```

```

lemma sorted-no-stutter-eq-distinct: sorted l ⇒ no-stutter l ⟷ distinct l
  ⟨proof⟩

```

```

definition distinct-ds :: 'a::linorder list ⇒ bool
  where distinct-ds l ≡ no-stutter (quicksort l)

```

```

lemma [code-unfold]: distinct = distinct-ds
  ⟨proof⟩

```

2.4.4 Parsing Rational Numbers

type-synonym $digit = nat$

Well-formedness conditions for a digit and a sequence of digits.

definition $wf_digit\ d \longleftrightarrow d < 10$

A sequence of digits is well-formed iff it is normalized and only contains well-formed digits.

definition $wf_digits\ ds \longleftrightarrow (\forall d \in set\ ds. wf_digit\ d)$

Functions to trim leading and trailing zeros.

fun $trim_ld_zs :: digit\ list \Rightarrow digit\ list$ **where**

$trim_ld_zs\ [] = []$
 $| trim_ld_zs\ (d\#\ ds) =$
 $(if\ d = 0\ then\ trim_ld_zs\ ds\ else\ d\#\ ds)$

lemma $tlz_hd_neg_z$: $trim_ld_zs\ ds = ds' \implies hd\ ds' \neq 0 \vee ds' = []$
 $\langle proof \rangle$

lemma $trim_ld_zs_idem$: $trim_ld_zs\ (trim_ld_zs\ ds) = trim_ld_zs\ ds$
 $\langle proof \rangle$

lemma $trim_ld_zs_wf$: $wf_digits\ ds \implies wf_digits\ (trim_ld_zs\ ds)$
 $\langle proof \rangle$

lemma $trim_ld_zs_app$: $trim_ld_zs\ (ds1\ @\ ds2) = (trim_ld_zs\ ds1)\ @\ ds2 \vee (\forall d \in set\ ds1. d = 0)$
 $\langle proof \rangle$

fun $trim_tr_zs' :: digit\ list \Rightarrow digit\ list \Rightarrow digit\ list$ **where**

$trim_tr_zs'\ []\ zs = []$
 $| trim_tr_zs'\ (d\#\ ds)\ zs =$
 $(if\ d = 0\ then\ trim_tr_zs'\ ds\ (0\#\ zs)$
 $else\ zs\ @\ d\ \# (trim_tr_zs'\ ds\ []))$

fun $trim_tr_zs :: digit\ list \Rightarrow digit\ list$ **where**

$trim_tr_zs\ ds = trim_tr_zs'\ ds\ []$

lemma $ttz_last_neg_z_aux$:
assumes $trim_tr_zs'\ ds\ zs = ds'$
shows $last\ ds' \neq 0 \vee ds' = []$
 $\langle proof \rangle$

lemma $ttz_last_neg_z$: $trim_tr_zs\ ds = ds' \implies last\ ds' \neq 0 \vee ds' = []$
 $\langle proof \rangle$

lemma $trim_tr_zs'_wf$: $wf_digits\ ds \wedge wf_digits\ zs \implies wf_digits\ (trim_tr_zs'\ ds\ zs)$
 $\langle proof \rangle$

lemma *trim-tr-zs-wf*: $wf_digits\ ds \implies wf_digits\ (trim_tr_zs\ ds)$
 <proof>

Functions to convert between integers and a sequence of digits.

fun *int-of-digits* :: *digit list* $\Rightarrow$ *int* $\Rightarrow$ *int* **where**
 int-of-digits [] *acc* = *acc*
 | *int-of-digits* (*d*#*ds*) *acc* = *int-of-digits* *ds* (*acc* * 10 + *d*)

fun *digits-of-int* :: *int* $\Rightarrow$ *digit list* **where**
 digits-of-int *i* =
 (if *i* $\leq$ 0 then []
 else if *i* < 10 then [nat *i*]
 else *digits-of-int* (*i* div 10) @ [nat (*i* mod 10)])

value *digits-of-int* (*int-of-digits* [1,2,3,4] 0)

value *int-of-digits* (*digits-of-int* 1234) 0

value *digits-of-int* (*int-of-digits* [1] 123)

lemma *digits-of-int-int-of-digits-aux*:
 assumes *acc* > 0 **and** $\forall d \in set\ ds.\ wf_digit\ d$
 shows *digits-of-int* (*int-of-digits* *ds* *acc*) = *digits-of-int* *acc* @ *ds*
 <proof>

Proof for correctness of function *int-of-digits* and *digits-of-int*.

lemma *digits-of-int-int-of-digits*:
 assumes *wf-digits* *ds*
 shows *digits-of-int* (*int-of-digits* *ds* 0) = *trim-ld-zs* *ds*
 <proof>

Function to convert from a sequences of digits to a rational number.

primrec *rat-of-digits-pair* :: *digit list* $\times$ *digit list* $\Rightarrow$ *rat* **where**
 rat-of-digits-pair (*ds*₁,*ds*₂) = (
 let *ds*₁' = *trim-ld-zs* *ds*₁; *ds*₂' = *trim-tr-zs* *ds*₂ in
 Rat.Fract (*int-of-digits* (*ds*₁' @ *ds*₂') 0) (10 ^{length} *ds*₂')
)

For proofs about the function *rat-of-digits-pair* see *Rat_Parsing.thy*.

2.4.5 Code Generation

export-code
 check-plan
 nat-of-integer integer-of-nat int-of-integer integer-of-int Inl Inr Rat.Fract Rat.of-int
rat-of-digits-pair
 predAtm eqAtm predicate Pred Func Either Var Obj PredDecl FuncDecl BigAnd
BigOr
 formula.Not formula.Bot Effect No-Const Time-Const Func-Const LEQ EQ GEQ

Simple-Action-Schema Durative-Action-Schema At-Start At-End Over-All
map-atom Domain Problem Simple-Plan-Action Durative-Plan-Action
term.CONST term.VAR
String.explode String.implode
in *SML*
module-name *TEMPORAL-PDDL-Checker-Exported*
file *code/TEMPORAL-PDDL-Checker-Exported.sml*

end — Theory

3 Connecting the Semantics to State-Transition Based Semantics

theory *TEMPORAL-PDDL-Semantics-Alt*
imports
 TEMPORAL-PDDL-Semantics
 TEMPORAL-PDDL-Checker
begin

Auxiliary lemma with about *dropWhile*, *takeWhile* with *le* and *leq*

lemma *strict-sorted-drop-le-take-leq-comm:*
 assumes $t_i \leq t_j$
 and *strict-sorted* (*map fst hs*)
 shows *dropWhile* (*le* t_i) (*takeWhile* (*leq* t_j) *hs*) = *takeWhile* (*leq* t_j) (*dropWhile* (*le* t_i) *hs*)
 <proof>

lemma *strict-sorted-drop-leq-take-le-comm:*
 assumes $t_i < t_j$
 and *strict-sorted* (*map fst hs*)
 shows *dropWhile* (*leq* t_i) (*takeWhile* (*le* t_j) *hs*) = *takeWhile* (*le* t_j) (*dropWhile* (*leq* t_i) *hs*)
 <proof>

lemma *strict-sorted-drop-leq-take-leq-comm:*
 assumes $t_i \leq t_j$
 and *strict-sorted* (*map fst hs*)
 shows *dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) *hs*) = *takeWhile* (*leq* t_j) (*dropWhile* (*leq* t_i) *hs*)
 <proof>

lemma *strict-sorted-drop-le-take-leq:*
 assumes *strict-sorted* (*map fst hs*)

and $(t_a, A) \in \text{set } hs$
shows $(t_a, A) \in \text{set } (\text{dropWhile } (le\ t_i) (\text{takeWhile } (leq\ t_j)\ hs)) \longleftrightarrow t_i \leq t_a \wedge t_a \leq t_j$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-drop-leq-take-leq*:
assumes *strict-sorted* $(\text{map fst } hs)$
and $(t_a, A) \in \text{set } hs$
shows $(t_a, A) \in \text{set } (\text{dropWhile } (leq\ t_i) (\text{takeWhile } (leq\ t_j)\ hs)) \longleftrightarrow t_i < t_a \wedge t_a \leq t_j$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-drop-leq-take-le*:
assumes *strict-sorted* $(\text{map fst } hs)$
and $(t_a, A) \in \text{set } hs$
shows $(t_a, A) \in \text{set } (\text{dropWhile } (leq\ t_i) (\text{takeWhile } (le\ t_j)\ hs)) \longleftrightarrow t_i < t_a \wedge t_a < t_j$
 $\langle \text{proof} \rangle$

lemma *split-list2-le*:
assumes *strict-sorted* xs
and $x_1 \in \text{set } xs$
and $x_2 \in \text{set } xs$
shows $x_1 < x_2 \longleftrightarrow (\exists\ xs_1\ xs_2\ xs_3. xs = xs_1 @ x_1 \# xs_2 @ x_2 \# xs_3)$
 $\langle \text{proof} \rangle$

lemma *drop-take-leq_i-leq-leq_j-aux*:
assumes $t_k \leq t_j$
and *strict-sorted* $(\text{map fst } hs)$
shows $\text{takeWhile } (leq\ t_k)\ hs @ \text{takeWhile } (leq\ t_j) (\text{dropWhile } (leq\ t_k)\ hs) = \text{takeWhile } (leq\ t_j)\ hs$
 $\langle \text{proof} \rangle$

lemma *drop-take-leq_i-leq-leq_j-simp*:
assumes $t_i \leq t_k$ **and** $t_k \leq t_j$
and *strict-sorted* $(\text{map fst } hs)$
shows $\text{dropWhile } (leq\ t_i) (\text{takeWhile } (leq\ t_k)\ hs) @ \text{dropWhile } (leq\ t_k) (\text{takeWhile } (leq\ t_j)\ hs) = \text{dropWhile } (leq\ t_i) (\text{takeWhile } (leq\ t_j)\ hs)$
 $\langle \text{proof} \rangle$

3.1 State-Transition Based Semantics

context *ast-problem* **begin**

fun *inst-cond* :: *ast-action-schema* $\Rightarrow$ *object list* $\Rightarrow$ *temporal-annotation* $\Rightarrow$ (*object atom*) *formula* **where**
inst-cond (*Durative-Action-Schema* n *params* d *cond* *eff*) *args* $ts =$

(let tsubst = subst-term (the o (map-of (zip (map fst params) args))) in
 (map-formula o map-atom) tsubst (BigAnd (filter-time-spec ts cond)))

fun res-inst-inv :: plan-action $\Rightarrow$ (object atom) formula option **where**
 res-inst-inv (Simple-Plan-Action n args) = None
 | res-inst-inv (Durative-Plan-Action n args d) =
 Some (inst-cond (the (resolve-action-schema n)) args Over-All)

lemma res-inst-inv-refine:

assumes wf-plan-action π
and Some a = res-inst-snap-action π Over-All
shows Some (precondition a) = res-inst-inv π
 <proof>

Set of all ground actions that, are executed at a given time point in the induced happening sequence.

definition acts-of-plan-at :: time $\Rightarrow$ plan $\Rightarrow$ ground-action set **where**
 acts-of-plan-at t_i πs =
 { a_π . $\exists \pi$. $(t_i, \pi) \in \text{simple-acts } \pi s \wedge \text{Some } a_\pi = \text{res-inst } \pi$ }
 $\cup$ { a_{start} . $\exists \pi$. $(t_i, \pi) \in \text{durative-acts } \pi s \wedge \text{Some } a_{start} = \text{res-inst-snap-action } \pi \text{ At-Start}$ }
 $\cup$ { a_{end} . $\exists t' \pi$. $(t', \pi) \in \text{durative-acts } \pi s \wedge t_i = t' + \text{duration } \pi \wedge \text{Some } a_{end} = \text{res-inst-snap-action } \pi \text{ At-End}$ }

Set of all invariant that have to hold in between consecutive happening time points $t' < t$.

definition invs-of-plan-at :: time $\Rightarrow$ plan $\Rightarrow$ (object atom) formula set **where**
 invs-of-plan-at t_i πs =
 {inv. $\exists t_\pi \pi$. $(t_\pi, \pi) \in \text{durative-acts } \pi s \wedge t_\pi < t_i \wedge t_i \leq t_\pi + (\text{duration } \pi) \wedge \text{Some inv} = \text{res-inst-inv } \pi$ }

Predicate for sequence of happening time points.

fun apply-eff :: ground-action set $\Rightarrow$ world-model $\Rightarrow$ world-model **where**
 apply-eff A_i $M = (M - \bigcup (\text{set 'dels' effect 'A}_i)) \cup \bigcup (\text{set 'adds' effect 'A}_i)$

Instead of explicitly defining an induced happening sequence. Here the happening sequence is only implicitly constructed. Only happening time points are considered for the valid execution, invariants are treated like preconditions of an entire happening.

fun valid-state-seq :: world-model $\Rightarrow$ time list $\Rightarrow$ plan $\Rightarrow$ world-model $\Rightarrow$ bool
where
 valid-state-seq $M \square \pi s M' \longleftrightarrow (M = M')$
 | valid-state-seq $M (t_i \# ts) \pi s M' \longleftrightarrow$
 (let $A_i = \text{acts-of-plan-at } t_i \pi s$ in
 ($\forall i \in \text{invs-of-plan-at } t_i \pi s$. $M \models i$)
 $\wedge (\forall a \in A_i$. $M \models \text{precondition } a$)
 $\wedge (\forall a \in A_i$. $\forall b \in A_i$. $a \neq b \longrightarrow \text{acts-non-intrf } a b$)

$\wedge \text{valid-state-seq } (\text{apply-eff } A_i \ M) \ ts \ \pi s \ M')$

definition *valid-plan-from* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *bool* **where**
 $\text{valid-plan-from } M \ \pi s \longleftrightarrow (\text{wf-plan } \pi s \wedge (\exists \text{https } M'. \text{https-seq } \pi s \ \text{https} \wedge$
 $\text{valid-state-seq } M \ \text{https } \pi s \ M' \wedge M' \models_{\text{c}} (\text{goal } P)))$

definition *valid-plan* :: *plan* $\Rightarrow$ *bool*
where $\text{valid-plan} \equiv \text{valid-plan-from } I$

Concise definition used in paper:

lemma $\text{valid-plan } \pi s \equiv \text{wf-plan } \pi s \wedge (\exists \text{https } M'. \text{https-seq } \pi s \ \text{https} \wedge \text{valid-state-seq}$
 $I \ \text{https } \pi s \ M' \wedge M' \models_{\text{c}} (\text{goal } P))$
 $\langle \text{proof} \rangle$

3.2 Connecting the State-Transition Based Semantics to Happening Sequence Based Semantics

Justification for Semantic refinement

lemma $\text{apply-happ } (t, A) \ M = \text{apply-eff } (\text{set } A) \ M$
 $\langle \text{proof} \rangle$

lemma *valid-state-seq-app-iff*:
 $\text{valid-state-seq } M_i \ (ts_1 @ ts_2) \ \pi s \ M_k$
 $\longleftrightarrow (\exists M_j. \text{valid-state-seq } M_i \ ts_1 \ \pi s \ M_j \wedge \text{valid-state-seq } M_j \ ts_2 \ \pi s \ M_k)$
 $\langle \text{proof} \rangle$

lemma *valid-state-seq-state-unique*:
assumes $\text{valid-state-seq } M_0 \ ts \ \pi s \ M_i$ **and** $\text{valid-state-seq } M_0 \ ts \ \pi s \ M_i'$
shows $M_i = M_i'$
 $\langle \text{proof} \rangle$

acts-of-plan-at returns exactly the set of actions that would be executed for a happening time point in the induced happening sequence

lemma *ind-happ-seq-htp-acts-of-plan-at*:
assumes $\text{ind-happ-seq } \pi s \ hs$
and $\text{is-htp } \pi s \ t$
and $(t, A) \in \text{set } hs$
shows $\text{set } A = \text{acts-of-plan-at } t \ \pi s$
 $\langle \text{proof} \rangle$
end

context *wf-ast-problem*
begin

The precondition of every invariant ground action in between two consecutive happening time points is included in *invs-of-plan-at*.

lemma *ind-happ-seq-invs-of-plan-at*:
assumes $\text{wf-plan } \pi s$

and *consec-htps* $\pi s \ t_i \ t_j$
and *ind-happ-seq* $\pi s \ hs$
and $(t, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \ hs))$
and $a \in \text{set } A$
shows $(\text{precondition } a) \in \text{invs-of-plan-at } t_j \ \pi s$
 $\langle \text{proof} \rangle$

Every invariant from *invs-of-plan-at* is placed as an invariant ground action in between two consecutive happening time points in the induced happening sequence.

lemma *invs-of-plan-at-ind-happ-seq*:
assumes *wf-plan* πs
and *consec-htps* $\pi s \ t_i \ t_j$
and *ind-happ-seq* $\pi s \ hs$
and $i \in \text{invs-of-plan-at } t_j \ \pi s$
shows $\exists (t, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \ hs)). \exists a \in \text{set } A. i = \text{precondition } a$
 $\langle \text{proof} \rangle$

In an induced happening sequence all ground actions between two consecutive happening time points have empty effect. Those ground actions are the instantiated durative action for the (**over all** ...) time specifier.

lemma *ind-happ-seq-leq_i-le_j-empty-eff*:
assumes *consec-htps* $\pi s \ t_i \ t_j$
and *wf-plan* πs
and *ind-happ-seq* $\pi s \ hs$
and $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \ hs))$
and $a \in \text{set } A$
shows $\text{effect } a = \text{Effect } [] \ []$
 $\langle \text{proof} \rangle$

Applying a happening containing only action with empty effect, doesn't change the current state.

lemma *apply-happ-empty-effect*:
assumes $\forall (t_a, A) \in \text{set } hs. \forall a \in \text{set } A. \text{effect } a = \text{Effect } [] \ []$
shows $\forall h \in \text{set } hs. \text{apply-happ } h \ M = M$
 $\langle \text{proof} \rangle$

Combining $\llbracket \text{wf-ast-problem } ?P; \text{consec-htps } ?\pi s \ ?t_i \ ?t_j; \text{ast-problem.wf-plan } ?P \ ?\pi s; \text{ast-problem.ind-happ-seq } ?P \ ?\pi s \ ?hs; (?t_a, ?A) \in \text{set } (\text{dropWhile } (\text{leq } ?t_i) (\text{takeWhile } (\text{le } ?t_j) \ ?hs)); ?a \in \text{set } ?A \rrbracket \implies \text{ground-action.effect } ?a = \text{Effect } [] \ []$ and $\forall (t_a, A) \in \text{set } ?hs. \forall a \in \text{set } A. \text{ground-action.effect } a = \text{Effect } [] \ [] \implies \forall h \in \text{set } ?hs. \text{apply-happ } h \ ?M = ?M$: Applying the happening in between two consecutive happening time points doesn't change the current state.

lemma *ind-happ-seq-leq_i-le_j*:
assumes *consec-htps* $\pi s \ t_i \ t_j$

and *wf-plan* πs
and *ind-happ-seq* πs hs
shows $\forall h \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)). \text{ apply-happ } h M$
 $= M$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq- M_i - M_i -precond*:
assumes $\forall h \in \text{set } hs. \text{ apply-happ } h M_i = M_i$
and *valid-happ-seq* M_i hs M_i
shows $\forall (t_a, as) \in \text{set } hs. \forall a \in \text{set } as. M_i \text{ }^c\models \text{precondition } a$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-leq $_i$ -le $_j$ -aux1*:
assumes *consec-htps* πs t_i t_j
and *wf-plan* πs
and *ind-happ-seq* πs hs
and *valid-happ-seq* M_i $(\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs))$ M_j
shows $\forall (t_a, as) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)). \forall a \in \text{set } as.$
 $M_i \text{ }^c\models \text{precondition } a$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-leq $_i$ -le $_j$ -aux2*:
assumes $\forall h \in \text{set } hs. \text{ apply-happ } h M_i = M_i$
and $\forall (t_a, A) \in \text{set } hs. \forall a \in \text{set } A. M_i \text{ }^c\models \text{precondition } a$
and $\forall (t_a, A) \in \text{set } hs. \forall (t_a', A') \in \text{set } hs.$
 $\forall a \in \text{set } A. \forall a' \in \text{set } A'. \text{ acts-non-intrf } a \ a'$
shows *valid-happ-seq* M_i hs M_i
 $\langle \text{proof} \rangle$

end — Context *wf-ast-problem*

context *ast-problem*
begin

All invariants from *invs-of-plan-at* are satisfied iff an induced happening sequence is valid in between two consecutive happening time points.

lemma (**in** *wf-ast-problem*) *valid-happ-seq-leq $_i$ -le $_j$ -invs-of-plan-at*:
assumes *wf-plan* πs
and *consec-htps* πs t_i t_j
and *ind-happ-seq* πs hs
shows *valid-happ-seq* M $(\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs))$ M
 $\longleftrightarrow (\forall i \in \text{invs-of-plan-at } t_j \ \pi s. M \text{ }^c\models i)$
 $\langle \text{proof} \rangle$

lemma *drop-take-leq $_i$ -le-leq $_j$ -aux*:
assumes $t_k \leq t_j$
and *strict-sorted* $(\text{map } \text{fst } hs)$
shows $\text{takeWhile } (\text{le } t_k) hs @ \text{takeWhile } (\text{leq } t_j) (\text{dropWhile } (\text{le } t_k) hs) =$
 $\text{takeWhile } (\text{leq } t_j) hs$

$\langle proof \rangle$

We can split up a happening sequence via *dropWhile* and *takeWhile*

lemma *drop-take-leq_i-le-leq_j-split-app*:
assumes $t_i < t_k$ **and** $t_k \leq t_j$
and *strict-sorted* (*map fst hs*)
shows $\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_k) \text{ hs}) @ \text{dropWhile } (\text{le } t_k) (\text{takeWhile } (\text{leq } t_j) \text{ hs})$
 $= \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) \text{ hs})$
 $\langle proof \rangle$

Some properties of *valid-happ-seq* that are needed later.

lemma *valid-happ-seq-app-iff*:
 $\text{valid-happ-seq } M_0 (\text{hs}_1 @ \text{hs}_2) M_j \longleftrightarrow (\exists M_i. \text{valid-happ-seq } M_0 \text{ hs}_1 M_i \wedge \text{valid-happ-seq } M_i \text{ hs}_2 M_j)$
 $\langle proof \rangle$

lemma *valid-happ-seq-split-iff*:
 $\text{valid-happ-seq } M_0 \text{ hs } M_j \longleftrightarrow (\exists M_i. \text{valid-happ-seq } M_0 (\text{takeWhile } f \text{ hs}) M_i \wedge \text{valid-happ-seq } M_i (\text{dropWhile } f \text{ hs}) M_j)$
 $\langle proof \rangle$

Every happening sequence contains a happening at every happening time point.

lemma *ind-happ-seq-happ-at-htps*:
assumes *is-htp* $\pi s \ t_i$
and *ind-happ-seq* $\pi s \ \text{hs}$
shows $\exists A_i. (t_i, A_i) \in \text{set } \text{hs}$
 $\langle proof \rangle$

lemma *distinct-single-set-list*: $\text{set } xs = \{x\} \implies \text{distinct } xs \implies xs = [x]$
 $\langle proof \rangle$

An induced happening sequence contain exactly one happening for every happening time point.

lemma *ind-happ-seq-le_i-leq_i*:
assumes *is-htp* $\pi s \ t_i$
and *ind-happ-seq* $\pi s \ \text{hs}$
shows $\exists A_i. \text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) \text{ hs}) = [(t_i, A_i)]$
 $\langle proof \rangle$

invs-of-plan-at is valid for a single happening time point iff any induced happening sequence is valid from the previous happening time point to the happening time point.

lemma (**in** *wf-ast-problem*) *valid-happ-seq-valid-state-seq-consec-htps*:
assumes *wf-plan* πs
and *consec-htps* $\pi s \ t_i \ t_j$

and *ind-happ-seq* πs hs
shows *valid-happ-seq* M (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs)) M'
 $\longleftrightarrow$ *valid-state-seq* M $[t_j]$ πs M'
 $\langle proof \rangle$

lemma *cons-consec-https*:
assumes *https-seq* πs $https$
and $https = ts_1 @ t_i \# t_j \# ts_2$
shows *consec-https* πs t_i t_j
 $\langle proof \rangle$

lemma (*in wf-ast-problem*) *valid-happ-seq-valid-state-seq-i-to-j-aux*:
fixes πs
assumes *https-seq* πs $https$
and $https = ts_1 @ t_i \# ts_2 @ t_j \# ts_3$
and *wf-plan* πs
and *ind-happ-seq* πs hs
shows *valid-happ-seq* M (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs)) M'
 $\longleftrightarrow$ *valid-state-seq* M ($ts_2 @ [t_j]$) πs M'
 $\langle proof \rangle$

lemma *prod-list-Nil*: $\forall x_1 x_2. (x_1, x_2) \notin set\ xs \implies xs = []$
 $\langle proof \rangle$

An induced happening sequence is only Nil iff the original plan was Nil

lemma *ind-happ-seq-Nil*: *ind-happ-seq* πs $hs \implies \pi s = [] \longleftrightarrow hs = []$
 $\langle proof \rangle$

lemma *https-seq-Nil-iff*:
assumes *https-seq* πs $https$
shows $\pi s = [] \longleftrightarrow https = []$
 $\langle proof \rangle$

lemma *non-Nil-sorted-last-max*:
assumes $(xs::('a::linorder)\ list) \neq []$
and *sorted* xs
and $x_{max} = last\ xs$
shows $\forall x \in set\ xs. x \leq x_{max}$
 $\langle proof \rangle$

Proof for the existence of a sequence of happening time point for every plan.
 Needed later in the proof for the completeness of the semantics.

lemma *https-seq-exists*: $\exists https. https-seq\ \pi s\ https$
 $\langle proof \rangle$

end

context *wf-ast-problem*

begin

Two induced happening sequences contain the same ground actions in between two consecutive happening time points.

lemma *ind-happ-seqs-same-acts-leq_i-le_j*:
assumes *consec-htps* πs t_i t_j
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_1))$
and $a \in \text{set } A$
shows $\exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_2)) \wedge a \in \text{set } A'$
<proof>

If one induced happening sequence is a valid happening sequence in between two consecutive happening time points, then every other induced happening sequence is also valid in between those consecutive happening time point.

lemma *valid-happ-seq-leq_i-le_j*:
assumes *consec-htps* πs t_i t_j
and *wf-plan* πs
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and *valid-happ-seq* M_i $(\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_1))$ M_j
shows *valid-happ-seq* M_i $(\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_2))$ M_j
<proof>

An induced happening sequence contain exactly one happening for every happening time point.

lemma *ind-happ-seq-le_i-leq_i*:
assumes *is-htp* πs t_i
and *ind-happ-seq* πs hs
shows $\exists A_i. \text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) hs) = [(t_i, A_i)]$
<proof>

Two induced happening sequences contain the same ground actions for every happening time point.

lemma *ind-happ-seqs-same-acts-le_i-leq_i*:
assumes *is-htp* πs t_i
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and $(t_a, A) \in \text{set } (\text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) hs_1))$
and $a \in \text{set } A$
shows $\exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) hs_2)) \wedge a \in \text{set } A'$
<proof>

With $\llbracket \text{is-htp } ?\pi s ?t_i; \text{ind-happ-seq } ?\pi s ?hs_1; \text{ind-happ-seq } ?\pi s ?hs_2; (?t_a, ?A) \in \text{set } (\text{dropWhile } (\text{le } ?t_i) (\text{takeWhile } (\text{leq } ?t_i) ?hs_1)); ?a \in \text{set } ?A \rrbracket \implies$

$\exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (\text{le } ?t_i) (\text{takeWhile } (\text{leq } ?t_i) ?hs_2)) \wedge ?a \in \text{set } A'$ we can prove: If we have one induced happening sequence that is a valid happening sequence for a single happening time point, then every other induced happening sequence is also valid for that happening time point.

lemma *valid-happ-seq-le_i-leq_i*:

assumes *is-htp* $\pi s \ t_i$
and *ind-happ-seq* $\pi s \ hs_1$
and *ind-happ-seq* $\pi s \ hs_2$
and *valid-happ-seq* $M_i \ (\text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) \ hs_1)) \ M_j$
shows *valid-happ-seq* $M_i \ (\text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) \ hs_2)) \ M_j$
<proof>

Putting together $\llbracket \text{wf-ast-problem } ?P; \text{consec-htps } ?\pi s \ ?t_i \ ?t_j; \text{ast-problem.wf-plan } ?P \ ?\pi s; \text{ast-problem.ind-happ-seq } ?P \ ?\pi s \ ?hs_1; \text{ast-problem.ind-happ-seq } ?P \ ?\pi s \ ?hs_2; \text{valid-happ-seq } ?M_i \ (\text{dropWhile } (\text{leq } ?t_i) (\text{takeWhile } (\text{le } ?t_j) \ ?hs_1)) \ ?M_j \rrbracket \implies \text{valid-happ-seq } ?M_i \ (\text{dropWhile } (\text{leq } ?t_i) (\text{takeWhile } (\text{le } ?t_j) \ ?hs_2)) \ ?M_j$ and $\llbracket \text{is-htp } ?\pi s \ ?t_i; \text{ind-happ-seq } ?\pi s \ ?hs_1; \text{ind-happ-seq } ?\pi s \ ?hs_2; \text{valid-happ-seq } ?M_i \ (\text{dropWhile } (\text{le } ?t_i) (\text{takeWhile } (\text{leq } ?t_i) \ ?hs_1)) \ ?M_j \rrbracket \implies \text{valid-happ-seq } ?M_i \ (\text{dropWhile } (\text{le } ?t_i) (\text{takeWhile } (\text{leq } ?t_i) \ ?hs_2)) \ ?M_j$. If we have one induced happening sequence that is valid from a happening time point to the next consecutive happening time point, that every other induced happening sequence is also valid for that interval.

lemma *valid-happ-seq-consec-htps*:

assumes *consec-htps* $\pi s \ t_i \ t_j$
and *wf-plan* πs
and *ind-happ-seq* $\pi s \ hs_1$
and *ind-happ-seq* $\pi s \ hs_2$
and *valid-happ-seq* $M_i \ (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) \ hs_1)) \ M_j$
shows *valid-happ-seq* $M_i \ (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) \ hs_2)) \ M_j$
<proof>

lemma *htps-state-trace-unique-i-to-j-aux*:

fixes πs
assumes *htps-seq* $\pi s \ htps$
and $htps = ts_1 \ @ \ t_i \ \# \ ts_2 \ @ \ t_j \ \# \ ts_3$
and *wf-plan* πs
and *ind-happ-seq* $\pi s \ hs_1$
and *ind-happ-seq* $\pi s \ hs_2$
and *valid-happ-seq* $M_i \ (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) \ hs_1)) \ M_j$
shows *valid-happ-seq* $M_i \ (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) \ hs_2)) \ M_j$
<proof>

With $\llbracket \text{wf-ast-problem } ?P; \text{consec-htps } ?\pi s \ ?t_i \ ?t_j; \text{ast-problem.wf-plan } ?P \ ?\pi s; \text{ast-problem.ind-happ-seq } ?P \ ?\pi s \ ?hs_1; \text{ast-problem.ind-happ-seq } ?P \ ?\pi s \ ?hs_2; \text{valid-happ-seq } ?M_i \ (\text{dropWhile } (\text{leq } ?t_i) (\text{takeWhile } (\text{leq } ?t_j) \ ?hs_1)) \ ?M_j \rrbracket \implies \text{valid-happ-seq } ?M_i \ (\text{dropWhile } (\text{leq } ?t_i) (\text{takeWhile } (\text{leq } ?t_j) \ ?hs_2)) \ ?M_j$

? M_j we can now proof that if one induced happening sequence is valid form a happening time point to another happening time point (not just consecutive), than every other induced happening sequence is valid for between those two happening time points.

lemma *valid-happ-seq-htp-i-to-htp-j*:

assumes $t_i < t_j$
and *is-htp* πs t_i
and *is-htp* πs t_j
and *wf-plan* πs
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and *valid-happ-seq* M_i (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs_1)) M_j
shows *valid-happ-seq* M_i (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs_2)) M_j
 $\langle proof \rangle$

lemma *happ-tp-leq-t_{min}-t_{max}*:

assumes $(t_\pi, \pi) \in set \pi s$
and *inst-of-plan-action* πs (t_π, π) (t_a, a)
shows $(\exists t_j. is-htp \pi s t_j \wedge t_j \leq t_a) \wedge (\exists t_j. is-htp \pi s t_j \wedge t_a \leq t_j)$
 $\langle proof \rangle$

Proof that every happening of a induced happening sequence is in between the smallest and largest happening time point.

lemma *ind-happ-seq-https-leq-t_{min}-t_{max}*:

fixes πs
assumes *ind-happ-seq* πs ($hs' @ hs$)
and $\forall t_i. is-htp \pi s t_i \longrightarrow t_{min} \leq t_i$
and $\forall t_i. is-htp \pi s t_i \longrightarrow t_i \leq t_{max}$
shows $\forall (t_a, A) \in set hs. t_{min} \leq t_a \wedge t_a \leq t_{max}$
 $\langle proof \rangle$

lemma *valid-happ-seq-state-unique*:

assumes *valid-happ-seq* M_0 hs M_i **and** *valid-happ-seq* M_0 hs M_i'
shows $M_i = M_i'$
 $\langle proof \rangle$

lemma *valid-happ-seq-iff-valid-state-seq*:

fixes πs
assumes *https-seq* πs $https$
and *wf-plan* πs
and *ind-happ-seq* πs hs
shows *valid-happ-seq* M_0 hs $M_n \longleftrightarrow valid-state-seq M_0 https \pi s M_n$
 $\langle proof \rangle$

lemma (in *wf-ast-problem*) *plan-happ-path-iff-valid-state-seq*:

assumes *wf-plan* πs **and** *https-seq* πs $https$
shows *plan-happ-path* M πs $M' \longleftrightarrow valid-state-seq M https \pi s M'$
 $\langle proof \rangle$

lemmas *plan-happ-path-iff-valid-state-seq-iff'*
 $= wf\text{-}ast\text{-}problem.plan\text{-}happ\text{-}path\text{-}iff\text{-}valid\text{-}state\text{-}seq[of\ P, OF\ wf\text{-}ast\text{-}problem.intro]$

lemma *strict-sorted xs $\implies$ strict-sorted ys $\implies$ set xs = set ys $\implies$ xs = ys*
 $\langle proof \rangle$

Proof that a sequence of happening time points is unique for every plan

lemma *https-seq-unique:*
assumes *https-seq πs https₁*
and *https-seq πs https₂*
shows *https₁ = https₂*
 $\langle proof \rangle$

lemma *valid-plan-from2-iff:*
assumes *wf-problem*
shows *valid-plan-from2 I $\pi s \longleftrightarrow$ valid-plan-from I πs*
 $\langle proof \rangle$

lemma
assumes *wf-problem*
shows *valid-plan2 $\pi s \longleftrightarrow$ valid-plan πs*
 $\langle proof \rangle$

end — Context of *ast-problem*

end — Theory

References

- [1] M. Abdulaziz and L. Koller. Formal semantics and formally verified validation for temporal planning. In *The 36th AAAI Conference on Artificial Intelligence (AAAI)*, 2022.
- [2] M. Abdulaziz and P. Lammich. AI Planning Languages Semantics. *Arch. Formal Proofs*, 2020.